

TRƯỜNG ĐẠI HỌC GIAO THÔNG VẬN TẢI
KHOA CÔNG NGHỆ THÔNG TIN



ĐỒ ÁN TỐT NGHIỆP
CỬ NHÂN

Đề tài:

XÂY DỰNG HỆ THỐNG NHẬN DIỆN VẬT
THỂ QUA CAMERA MẮT CÁ

Sinh viên thực hiện	Nguyễn Quốc Nam
Mã số sinh viên	221220938
Lớp	CNTT1-K63
Khóa	63
Ngành đào tạo	Công nghệ thông tin
Hệ đào tạo	Chính quy
Giảng viên hướng dẫn	TS. Nguyễn Đức Dư

Hà Nội – 2026

TRƯỜNG ĐẠI HỌC GIAO THÔNG VẬN TẢI
KHOA CÔNG NGHỆ THÔNG TIN

ĐỒ ÁN TỐT NGHIỆP
CỬ NHÂN

Đề tài:

XÂY DỰNG HỆ THỐNG NHẬN DIỆN VẬT
THỂ QUA CAMERA MẮT CÁ

Sinh viên thực hiện	Nguyễn Quốc Nam
Mã số sinh viên	221220938
Lớp	CNTT1-K63
Khóa	63
Ngành đào tạo	Công nghệ thông tin
Hệ đào tạo	Chính quy
Giảng viên hướng dẫn	TS. Nguyễn Đức Dư

Hà Nội – 2026

LỜI CẢM ƠN

Lời đầu tiên, tôi xin bày tỏ lòng biết ơn sâu sắc đến TS. Nguyễn Đức Dư – giảng viên hướng dẫn đã tận tình chỉ bảo, định hướng nghiên cứu và động viên tôi trong suốt quá trình thực hiện đồ án tốt nghiệp này. Những góp ý chuyên môn và kinh nghiệm thực tiễn của thầy đã giúp tôi vượt qua nhiều khó khăn trong việc triển khai mô hình học sâu trên dữ liệu camera fisheye.

Tôi cũng xin gửi lời cảm ơn chân thành đến quý thầy cô trong Khoa Công nghệ Thông tin – Trường Đại học Giao thông Vận tải đã truyền đạt kiến thức nền tảng vững chắc trong suốt bốn năm học tập, đặc biệt các học phần Thị giác Máy tính, Học sâu và Xử lý Ảnh số, làm cơ sở lý thuyết cho đồ án này.

Xin cảm ơn ban quản trị cộng đồng mã nguồn mở Ultralytics và các nhóm nghiên cứu đã công bố bộ dữ liệu FishEye8K (ECCV 2023 Workshop), VisDrone2019 (IEEE/CVF ICCV 2019 Workshop) cùng thư viện SAHI – những tài nguyên quý giá tạo nền tảng thực nghiệm cho đề tài.

Cuối cùng, tôi xin gửi lời cảm ơn đặc biệt đến gia đình và bạn bè đã luôn đồng hành, chia sẻ và là nguồn động lực to lớn để tôi hoàn thành đồ án đúng hạn.

Do thời gian nghiên cứu có hạn và đây là lần đầu tiên thực hiện đề tài ở quy mô đồ án tốt nghiệp, chắc chắn báo cáo không tránh khỏi những thiếu sót. Kính mong quý thầy cô và các bạn đọc góp ý để tôi hoàn thiện hơn trong các nghiên cứu tiếp theo.

Hà Nội, ngày 1 tháng 6 năm 2026

Sinh viên thực hiện

MỤC LỤC

LỜI CẢM ƠN.....	i
MỤC LỤC	ii
DANH MỤC TỪ VIẾT TẮT	vi
DANH MỤC BẢNG BIỂU	viii
DANH MỤC HÌNH VẼ	ix
MỞ ĐẦU	1
CHƯƠNG 1. TỔNG QUAN VỀ ĐỀ TÀI VÀ CƠ SỞ LÝ THUYẾT	3
1.1. Bối cảnh và tính cấp thiết của đề tài	3
1.1.1. Thực trạng giao thông đô thị tại Việt Nam	3
1.1.2. Xu hướng ứng dụng AI trong giám sát giao thông	3
1.1.3. Vai trò của camera fisheye trong hệ thống giám sát	4
1.2. Tổng quan về phát hiện đối tượng trong thị giác máy tính	4
1.2.1. Giới thiệu bài toán phát hiện đối tượng	4
1.2.2. Phân loại các phương pháp phát hiện đối tượng	5
1.2.3. Các thách thức đặc thù trong môi trường camera fisheye	6
1.3. Camera Fisheye – Mô hình hình học và đặc điểm kỹ thuật	6
1.3.1. Nguyên lý quang học của ống kính fisheye	6
1.3.2. Các mô hình chiếu fisheye phổ biến	7
1.3.3. Biến dạng hình học và ảnh hưởng đến phát hiện đối tượng	8
1.3.4. Hàm biến đổi fisheye sử dụng trong đề tài	9
1.3.5. Chuyển đổi bounding box sang không gian fisheye	9
1.4. Kiến trúc YOLOv11 và lịch sử phát triển họ YOLO	10
1.4.1. Lịch sử phát triển các thế hệ YOLO	10
1.4.2. Kiến trúc tổng thể YOLOv11	11
1.4.3. Backbone C3k2 – Cross Stage Partial	12
1.4.4. Module AIFI Transformer trong Neck	12
1.4.5. Detection Head anchor-free	13
1.4.6. So sánh YOLOv11 với các phiên bản trước	13
1.5. Kỹ thuật SAHI – Sliced Aided Hyper Inference	13

1.5.1. Vấn đề phát hiện đối tượng nhỏ trong ảnh độ phân giải cao	13
1.5.2. Nguyên lý hoạt động của SAHI	14
1.5.3. Chiến lược tổng hợp kết quả và NMM	14
1.5.4. Ứng dụng SAHI trong bài toán fisheye	15
1.6. Hàm mất mát và chiến lược tối ưu hóa	15
1.6.1. CIoU Loss cho hồi quy bounding box	15
1.6.2. Distribution Focal Loss (DFL)	16
1.6.3. Chiến lược tối ưu hóa (AdamW/SGD) và cosine annealing	16
1.7. Kỹ thuật tăng cường dữ liệu	16
1.7.1. Mosaic Augmentation v2	16
1.7.2. MixUp Augmentation	17
1.7.3. Copy-Paste Augmentation	17
1.8. Các bộ dữ liệu liên quan	18
1.8.1. Bộ dữ liệu FishEye8K	18
1.8.2. Bộ dữ liệu VisDrone2019	19
1.8.3. So sánh và lý do lựa chọn	19
1.9. Tổng quan các công trình nghiên cứu liên quan	20
1.9.1. Phát hiện đối tượng trên ảnh fisheye	20
1.9.2. Phát hiện đối tượng nhỏ từ góc nhìn UAV	20
1.9.3. Khoảng trống nghiên cứu và đóng góp của đề tài	21
1.10. Phát biểu bài toán và mục tiêu nghiên cứu	21
1.10.1. Phát biểu bài toán	21
1.10.2. Mục tiêu và phạm vi nghiên cứu	22
1.10.3. Phương pháp nghiên cứu	22
1.10.4. Công nghệ và công cụ sử dụng	23
1.10.5. Quy trình phát triển	27
CHƯƠNG 2. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG	28
2.1. Đặc tả yêu cầu hệ thống	28
2.1.1. Khảo sát hệ thống camera giao thông trực tuyến	28
2.1.2. Yêu cầu chức năng	32

2.1.3. Yêu cầu phi chức năng	34
2.2. Kiến trúc tổng thể hệ thống	35
2.2.1. Kiến trúc phân lớp	35
2.2.2. Kiến trúc xử lý video bất đồng bộ	37
2.2.3. Luồng dữ liệu	38
2.3. Thiết kế cơ sở dữ liệu	38
2.3.1. Bảng dữ liệu	38
2.3.2. Chiến lược tương thích đa CSDL	39
2.4. Thiết kế REST API	40
2.4.1. Quy ước thiết kế API	40
2.4.2. Danh sách API endpoint	40
2.5. Thiết kế giao diện người dùng	42
CHƯƠNG 3. HUẤN LUYỆN VÀ ĐÁNH GIÁ MÔ HÌNH	43
3.1. Bộ dữ liệu sử dụng	43
3.1.1. Bộ dữ liệu FishEye8K	43
3.1.2. Bộ dữ liệu VisDrone2019	44
3.2. Tiền xử lý và bổ sung dữ liệu	45
3.2.1. Pipeline chuyển đổi VisDrone sang fisheye	45
3.2.2. Cân bằng dữ liệu theo lớp	46
3.3. Cấu hình huấn luyện	47
3.3.1. Môi trường huấn luyện	47
3.3.2. Siêu tham số huấn luyện	47
3.3.3. Cấu trúc file checkpoint	48
3.4. Kết quả huấn luyện	48
3.4.1. Quá trình hội tụ	48
3.4.2. Kết quả đánh giá trên tập kiểm định	49
3.5. So sánh hai phiên bản YOLOv11-N	51
CHƯƠNG 4. XÂY DỰNG HỆ THỐNG GIÁM SÁT GIAO THÔNG	52
4.1. Kiến trúc ứng dụng Flask	52
4.1.1. Cấu trúc thư mục dự án	52

4.1.2. Khởi tạo ứng dụng Flask	53
4.1.3. Xử lý concurrent requests	54
4.2. Module ước lượng tốc độ phương tiện (SpeedEstimator)	54
4.2.1. Nguyên lý theo dõi IoU	54
4.2.2. Chuyển đổi pixel displacement → tốc độ km/h	54
4.3. Module phát hiện tắc nghẽn giao thông (CongestionDetector)	55
4.3.1. Phương pháp phân tích mật độ ROI	55
4.3.2. Hiển thị trực quan	55
4.4. Module phân tích luồng giao thông (Analytics)	55
4.5. Giao diện người dùng và kiểm thử	56
4.5.1. Giao diện web	56
4.5.2. Kiểm thử chức năng	57
4.5.3. Đánh giá hiệu năng tổng thể	59
KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	60
TÀI LIỆU THAM KHẢO	62

DANH MỤC TỪ VIẾT TẮT

Từ viết tắt	Giải thích
AI	Artificial Intelligence – Trí tuệ nhân tạo
AMP	Automatic Mixed Precision – Độ chính xác hỗn hợp tự động
API	Application Programming Interface – Giao diện lập trình ứng dụng
BBox	Bounding Box – Hộp giới hạn đối tượng
CNN	Convolutional Neural Network – Mạng nơ-ron tích chập
CPU	Central Processing Unit – Bộ xử lý trung tâm
CV	Computer Vision – Thị giác máy tính
DFL	Distribution Focal Loss – Hàm mất mát phân phối tiêu điểm
FishEye8K	Bộ dữ liệu fisheye 8.000 ảnh từ cuộc thi AI City Challenge
FPN	Feature Pyramid Network – Mạng kim tự tháp đặc trưng
FPS	Frames Per Second – Số khung hình mỗi giây
GCS	Google Cloud Storage – Dịch vụ lưu trữ đám mây Google
GFLOPs	Giga Floating-Point Operations Per Second
GPU	Graphics Processing Unit – Bộ xử lý đồ họa

IoU	Intersection over Union – Chỉ số giao thoa trên hợp nhất
mAP	mean Average Precision – Độ chính xác trung bình tổng hợp
NMS	Non-Maximum Suppression – Triệt tiêu phi cực đại
ONNX	Open Neural Network Exchange – Định dạng mô hình mở
REST	Representational State Transfer – Kiến trúc dịch vụ web
ROI	Region of Interest – Vùng quan tâm
SAHI	Sliced Aided Hyper Inference – Suy diễn tăng cường chia lát
UAV	Unmanned Aerial Vehicle – Máy bay không người lái
VisDrone	Bộ dữ liệu phát hiện đối tượng từ UAV (IEEE/CVF 2019)
YOLO	You Only Look Once – Kiến trúc phát hiện đối tượng thời gian thực

DANH MỤC BẢNG BIỂU

Bảng 1.1. So sánh các phương pháp phát hiện đối tượng theo paradigm	5
Bảng 1.2. Các mô hình chiếu fisheye phổ biến và đặc điểm	7
Bảng 1.3. So sánh các thể hệ YOLO từ v1 đến v11	10
Bảng 1.4. Thống kê bộ dữ liệu FishEye8K theo tập train/val/test	18
Bảng 1.5. Thống kê bộ dữ liệu VisDrone2019	19
Bảng 1.6. Công nghệ và công cụ sử dụng	23
Bảng 2.1. So sánh hệ thống camera giao thông hiện có với hệ thống đề xuất	32
Bảng 2.2. Danh sách yêu cầu chức năng hệ thống	32
Bảng 2.3. Cấu trúc các bảng chính trong CSDL	38
Bảng 2.4. Danh sách API endpoint và mô tả chức năng	40
● Bảng thống kê real-time: số phương tiện hiện tại theo lớp, FPS xử lý, mức độ ùn tắc.	42
Bảng 3.1. Thống kê bộ dữ liệu FishEye8K	43
Bảng 3.2. Thống kê bộ dữ liệu VisDrone2019	44
Bảng 3.3. Ảnh xạ lớp đối tượng từ VisDrone sang FishEye8K	44
Bảng 3.4. Thống kê bộ dữ liệu sau khi gộp FishEye8K + VisDrone-fisheye	46
Bảng 3.5. Siêu tham số huấn luyện YOLOv11-N	48
Bảng 3.6. Kết quả huấn luyện – Precision, Recall, mAP theo từng lớp	49
Bảng 3.7. So sánh YOLOv11-N Cơ bản và YOLOv11-N Nâng cao	51
Bảng 3.7 so sánh trực quan hiệu năng tổng thể giữa hai phiên bản YOLOv11-N Cơ bản và Nâng cao. Kết quả cho thấy phiên bản Nâng cao mang lại hiệu quả vượt trội trên mọi khía cạnh học thuật: mAP@0.5 tăng từ 0.619 lên 0.862 (+39.3%), mAP@0.5:0.95 tăng từ 0.363 lên 0.572 (+57.6%), Precision tăng từ 0.650 lên 0.869 (+33.7%), và Recall tăng từ 0.570 lên 0.762 (+33.7%). Sự cải thiện vượt bậc này chứng minh tính đúng đắn của chiến lược bổ sung dữ liệu chuyển đổi từ VisDrone và đóng băng backbone để tránh overfitting.	
Bảng 4.1. Danh sách module và chức năng trong package fisheye_demo	52
Bảng 4.2. Bốn mức độ tắc nghẽn giao thông	55
Bảng 4.3. Kết quả kiểm thử chức năng hệ thống	58

DANH MỤC HÌNH VẼ

Hình 1.1. Camera fisheye 360° lắp đặt tại nút giao thông đô thị	7
Hình 2.1. Giao diện của hệ thống camera.0511.vn	29
Hình 2.2. Giao diện của hệ thống camera alltrafficcams.com	31
Hình 2.3. Sơ đồ kiến trúc hệ thống tổng thể	36
Hình 2.4. Kiến trúc xử lý video bất đồng bộ	38
Hình 3.1. Mẫu ảnh từ bộ dữ liệu FishEye8K với nhãn bounding box	43
Hình 3.2. Mẫu ảnh từ bộ dữ liệu VisDrone2019 (góc nhìn UAV)	45
Hình 3.3. Pipeline chuyển đổi VisDrone → fisheye và gộp dataset	46
Hình 3.4. Đường cong training loss và validation loss theo epoch	49
Hình 3.5. Đường cong mAP@0.5 và mAP@0.5:0.95 theo epoch	49
Hình 4.1. Giao diện web tổng quan hệ thống giám sát giao thông	57
Hình 4.2. Giao diện tải lên video và xem kết quả phát hiện đối tượng	57

MỞ ĐẦU

1. Lý do chọn đề tài

Trong bối cảnh đô thị hóa nhanh chóng và sự gia tăng đột biến của phương tiện giao thông tại các thành phố lớn Việt Nam, nhu cầu xây dựng các hệ thống giám sát giao thông thông minh ngày càng trở nên cấp thiết. Theo số liệu thống kê công bố năm 2024 của Tổng cục Thống kê Việt Nam, cả nước có hơn 7,8 triệu ô tô và 73 triệu xe máy đang lưu hành, gây ra áp lực lớn lên hạ tầng giao thông đô thị [24]. Số vụ tai nạn giao thông trong năm 2023 vẫn còn ở mức đáng lo ngại với hơn 10.000 vụ tai nạn nghiêm trọng được ghi nhận.

Camera giám sát giao thông hiện đại, đặc biệt là camera fisheye (camera mắt cá), ngày càng được ưa chuộng trong các ứng dụng giám sát đô thị do khả năng bao phủ góc nhìn rất rộng (lên đến 180° – 220°), cho phép quan sát toàn bộ một nút giao thông chỉ với một thiết bị duy nhất. Tuy nhiên, hình ảnh từ camera fisheye bị biến dạng méo hình thùng (barrel distortion) đặc trưng – các đường thẳng trong thực tế xuất hiện cong vênh và tỷ lệ kích thước đối tượng thay đổi không tuyến tính theo khoảng cách tâm ảnh – khiến các mô hình phát hiện đối tượng truyền thống huấn luyện trên ảnh thẳng (perspective camera) không thể áp dụng trực tiếp mà không có bước tiền xử lý hoặc tinh chỉnh đặc biệt [6].

Sự ra đời của kiến trúc YOLOv11 (Ultralytics, 2024) với backbone C3k2 cải tiến, module AIFI transformer và detection head anchor-free mở ra cơ hội triển khai hệ thống phát hiện đối tượng thời gian thực có độ chính xác cao trên dữ liệu fisheye [3]. Kết hợp với kỹ thuật SAHI (Sliced Aided Hyper Inference) [4] cho phép nâng cao đáng kể khả năng phát hiện các đối tượng nhỏ như người đi bộ ở xa tâm ảnh, đề tài này hướng đến xây dựng một hệ thống nhận diện vật thể hoàn chỉnh, ứng dụng thực tế trong giám sát giao thông đô thị Việt Nam.

2. Mục tiêu đề tài

Đề tài hướng đến xây dựng một hệ thống nhận diện vật thể đầy đủ trên camera fisheye sử dụng kiến trúc YOLOv11, bao gồm: (i) fine-tune mô hình YOLOv11-N trên bộ dữ liệu fisheye kết hợp FishEye8K và VisDrone2019 đã chuyển đổi; (ii) tích hợp kỹ thuật SAHI để nâng cao khả năng phát hiện đối tượng nhỏ; (iii) xây dựng ứng dụng Flask hoàn chỉnh với các module phân tích giao thông thông minh.

3. Đối tượng và phạm vi nghiên cứu

Đối tượng nghiên cứu: Mô hình phát hiện đối tượng YOLOv11-N, kỹ thuật SAHI, dữ liệu ảnh từ camera fisheye. Phạm vi: phát hiện 5 lớp đối tượng giao thông (Car, Bus, Truck, Pedestrian, Motorbike) trong điều kiện camera fisheye góc nhìn từ trên cao.

4. Phương pháp nghiên cứu

Kết hợp nghiên cứu lý thuyết (mô hình hình học fisheye, kiến trúc deep learning) với thực nghiệm (fine-tuning, đánh giá mAP, kiểm thử hệ thống). Dữ liệu thực nghiệm từ hai bộ dữ liệu công khai: FishEye8K [20] và VisDrone2019 [5].

5. Cấu trúc đồ án

Đồ án được tổ chức thành bốn chương chính: Chương 1 trình bày tổng quan và toàn bộ cơ sở lý thuyết làm nền tảng cho đề tài; Chương 2 phân tích và thiết kế hệ thống; Chương 3 mô tả quá trình huấn luyện và đánh giá mô hình; Chương 4 trình bày việc xây dựng ứng dụng giám sát giao thông thông minh. Phần kết luận tổng kết kết quả và định hướng phát triển.

CHƯƠNG 1. TỔNG QUAN VỀ ĐỀ TÀI VÀ CƠ SỞ LÝ THUYẾT

1.1. Bối cảnh và tính cấp thiết của đề tài

1.1.1. Thực trạng giao thông đô thị tại Việt Nam

Việt Nam đang trải qua quá trình đô thị hóa với tốc độ thuộc hàng nhanh nhất khu vực Đông Nam Á. Dân số đô thị đã vượt 38% tổng dân số cả nước và tiếp tục tăng nhanh theo từng năm. Kéo theo đó, số lượng phương tiện giao thông cơ giới tăng theo cấp số nhân: theo số liệu thống kê công bố năm 2024 của Tổng cục Thống kê Việt Nam, cả nước đang có hơn 7,8 triệu ô tô và 73 triệu xe máy được đăng ký lưu hành [24]. Riêng tại Hà Nội và Thành phố Hồ Chí Minh, mật độ phương tiện tại các nút giao thông trọng yếu vào giờ cao điểm thường xuyên vượt ngưỡng tắc nghẽn nghiêm trọng.

Thực trạng này đặt ra áp lực lớn lên hệ thống hạ tầng giao thông đô thị vốn đã quá tải. Các hệ thống camera giám sát giao thông tuy đã được triển khai rộng rãi tại các thành phố lớn, nhưng phần lớn chỉ phục vụ mục đích quan sát trực tiếp bởi nhân lực tại trung tâm điều hành, thiếu hoàn toàn khả năng phân tích tự động và cảnh báo thông minh. Theo đó, việc phát triển một hệ thống giám sát giao thông ứng dụng trí tuệ nhân tạo, có thể tự động phát hiện phương tiện, ước lượng mật độ, nhận diện ùn tắc và đưa ra cảnh báo kịp thời là nhu cầu cấp thiết và có giá trị thực tiễn cao.

1.1.2. Xu hướng ứng dụng AI trong giám sát giao thông

Trên thế giới, việc ứng dụng trí tuệ nhân tạo và thị giác máy tính (Computer Vision) vào hệ thống giao thông thông minh (Intelligent Transportation Systems – ITS) đã trở thành xu hướng chủ đạo trong thập kỷ qua. Các thành phố lớn tại Trung Quốc, Singapore, Hoa Kỳ và châu Âu đã triển khai thành công các hệ thống giám sát giao thông tự động có khả năng nhận diện phương tiện, phân tích luồng giao thông theo thời gian thực và phối hợp điều khiển đèn tín hiệu để giảm thiểu tắc nghẽn.

Các thuật toán phát hiện đối tượng dựa trên học sâu (deep learning), đặc biệt là họ mô hình YOLO (You Only Look Once), đã chứng minh được hiệu quả vượt trội so với các phương pháp truyền thống về cả tốc độ xử lý lẫn độ chính xác [1]. Mô hình YOLO từ phiên bản đầu tiên công bố năm 2016 đã liên tục được cải tiến qua nhiều thế hệ, đến nay phiên bản YOLOv11 (2024) đã đạt đến mức độ chính xác và tốc độ phù hợp cho triển khai thực tế trong các hệ thống nhúng và edge computing [3].

Tại Việt Nam, nghiên cứu và triển khai AI trong giám sát giao thông còn ở giai đoạn ban đầu. Phần lớn các đề tài nghiên cứu tập trung vào camera perspective thông thường với bộ dữ liệu huấn luyện từ nước ngoài, chưa quan tâm đúng mức đến đặc thù

của camera fisheye – loại camera đang được ứng dụng ngày càng phổ biến trong các hệ thống giám sát thực tế do khả năng bao phủ góc nhìn rộng.

1.1.3. Vai trò của camera fisheye trong hệ thống giám sát

Camera fisheye sở hữu nhiều ưu điểm vượt trội so với camera perspective thông thường trong các ứng dụng giám sát giao thông. Với góc nhìn lên đến 180° – 220° , một camera fisheye có thể bao quát toàn bộ một nút giao thông, bao gồm tất cả các làn xe và khu vực dành cho người đi bộ, trong khi một camera perspective thông thường thường chỉ quan sát được một phạm vi hẹp và cần nhiều thiết bị để bao phủ cùng diện tích đó [6].

Ngoài tiết kiệm chi phí lắp đặt và bảo trì, camera fisheye còn cho phép theo dõi quỹ đạo di chuyển của phương tiện trên một góc nhìn tổng thể, thuận tiện cho việc phân tích hành vi giao thông và phát hiện vi phạm. Đây là lý do tại sao camera fisheye ngày càng được các đô thị thông minh ưa chuộng trong thiết kế hệ thống giám sát giao thông thế hệ mới.

Tuy nhiên, đặc điểm méo hình thùng (barrel distortion) của camera fisheye tạo ra thách thức kỹ thuật đáng kể: các mô hình phát hiện đối tượng được huấn luyện trên ảnh perspective thông thường không thể hoạt động hiệu quả trên ảnh fisheye mà không có bước xử lý đặc biệt. Hơn nữa, bộ dữ liệu huấn luyện chuyên biệt cho camera fisheye còn khan hiếm so với các bộ dữ liệu perspective như COCO hay ImageNet, khiến việc nghiên cứu và phát triển mô hình phát hiện đối tượng trên fisheye trở thành một bài toán thú vị và có giá trị thực tiễn cao [7].

1.2. Tổng quan về phát hiện đối tượng trong thị giác máy tính

1.2.1. Giới thiệu bài toán phát hiện đối tượng

Phát hiện đối tượng (Object Detection) là một trong những bài toán cơ bản và quan trọng nhất trong lĩnh vực thị giác máy tính. Bài toán yêu cầu hệ thống không chỉ nhận diện được sự hiện diện của các đối tượng thuộc các lớp được định nghĩa trước trong một ảnh đầu vào, mà còn xác định chính xác vị trí của từng đối tượng thông qua một hộp giới hạn (bounding box) bao quanh đối tượng đó.

Về mặt hình thức, cho một ảnh đầu vào I có kích thước $H \times W \times C$, bài toán phát hiện đối tượng yêu cầu tìm tập hợp $D = \{(b_i, c_i, s_i)\}$ trong đó $b_i = (x, y, w, h)$ là tọa độ bounding box của đối tượng thứ i , $c_i \in \{1, \dots, K\}$ là nhãn lớp và $s_i \in [0, 1]$ là điểm tin cậy (confidence score). Để đánh giá độ chính xác của các bounding box dự đoán, chỉ số Intersection over Union (IoU) được sử dụng phổ biến [44]:

$$IoU = \frac{|B_{pred} \cap B_{gt}|}{|B_{pred} \cup B_{gt}|}$$

trong đó B_{pred} là bounding box dự đoán và B_{gt} là bounding box ground-truth. Một dự đoán được coi là đúng (True Positive) khi IoU vượt ngưỡng định trước, thường là 0,5.

Chỉ số đánh giá tổng hợp được dùng phổ biến nhất trong cộng đồng nghiên cứu là mean Average Precision (mAP). $mAP@0.5$ tính trung bình AP của tất cả các lớp đối tượng tại ngưỡng IoU = 0,5, trong khi $mAP@0.5:0.95$ tính trung bình trên nhiều ngưỡng IoU từ 0,5 đến 0,95 với bước nhảy 0,05, phản ánh độ chính xác định vị bounding box toàn diện hơn [44].

1.2.2. Phân loại các phương pháp phát hiện đối tượng

Lịch sử phát triển của phát hiện đối tượng có thể chia thành ba giai đoạn chính:

Giai đoạn trước deep learning (trước 2012): Các phương pháp truyền thống như Viola-Jones, Histogram of Oriented Gradients (HOG) kết hợp Support Vector Machine (SVM) và Deformable Part Models (DPM) dựa trên đặc trưng thủ công (hand-crafted features). Các phương pháp này bị giới hạn về độ chính xác và khả năng tổng quát hóa.

Giai đoạn two-stage detectors (2014–2016): Region-based CNN (R-CNN) [9] và các biến thể Fast R-CNN, Faster R-CNN đã mang lại bước đột phá về độ chính xác bằng cách tách biệt bước đề xuất vùng quan tâm (Region Proposal Network) và bước phân loại. Tuy nhiên, kiến trúc hai giai đoạn này đòi hỏi chi phí tính toán lớn, không phù hợp cho xử lý thời gian thực.

Giai đoạn one-stage detectors (từ 2016): YOLO [1], SSD [10], RetinaNet [23] và EfficientDet [17] xử lý toàn bộ ảnh trong một lần forward pass duy nhất, đánh đổi một phần độ chính xác để đổi lấy tốc độ xử lý nhanh hơn nhiều lần. Đây là nền tảng cho các ứng dụng real-time hiện đại.

Bảng 1.1. So sánh các phương pháp phát hiện đối tượng theo paradigm

Paradigm	Đại diện	Ưu điểm	Nhược điểm
Two-stage	Faster R-CNN	Độ chính xác cao	Chậm, không phù hợp realtime
One-stage	YOLO, SSD, EfficientDet	Nhanh, phù hợp realtime	Kém hơn với vật thể nhỏ
Transformer-based	DETR [16]	Loại bỏ NMS, end-to-end	Chi phí tính toán lớn

Paradigm	Đại diện	Ưu điểm	Nhược điểm
Anchor-free	CenterNet [42], YOLOv11 [3]	Đơn giản hóa thiết kế	Cần chiến lược gán nhãn tốt

1.2.3. Các thách thức đặc thù trong môi trường camera fisheye

Phát hiện đối tượng trên ảnh fisheye gặp phải những thách thức đặc thù so với ảnh perspective thông thường:

Biến dạng hình học phi tuyến: Barrel distortion làm cho hình dạng của phương tiện bị méo không đồng đều trên toàn bộ khung hình. Xe ô tô ở trung tâm ảnh gần như không bị biến dạng, nhưng cùng loại xe đó ở vùng biên ảnh có thể bị méo đến mức khó nhận ra. Điều này làm cho bounding box hình chữ nhật truyền thống trở nên kém phù hợp để bao chứa các đối tượng ở vùng biên [7].

Đối tượng nhỏ và mật độ cao: Với góc nhìn rộng, camera fisheye nhìn thấy nhiều đối tượng hơn trong cùng một khung hình nhưng mỗi đối tượng lại chiếm diện tích pixel nhỏ hơn. Người đi bộ ở xa tâm ảnh có thể chỉ chiếm 10–30 pixel chiều cao, thách thức khả năng phát hiện của các mô hình được huấn luyện cho đối tượng có kích thước chuẩn [4].

Phân phối kích thước không đồng đều: Đối tượng gần tâm ảnh và đối tượng ở rìa ảnh có tỷ lệ kích thước rất khác nhau do đặc tính phóng đại của ống kính. Mô hình cần học được invariant này để hoạt động tốt trên toàn bộ vùng ảnh.

Thiếu dữ liệu huấn luyện: Số lượng bộ dữ liệu fisheye có nhãn annotation chất lượng cao còn rất hạn chế so với các bộ dữ liệu perspective như COCO (330.000 ảnh) hay Open Images. Bộ dữ liệu fisheye lớn nhất hiện có là FishEye8K (8.000 ảnh) [20] vẫn còn khá nhỏ cho việc huấn luyện từ đầu.

1.3. Camera Fisheye – Mô hình hình học và đặc điểm kỹ thuật

1.3.1. Nguyên lý quang học của ống kính fisheye

Ống kính fisheye là loại ống kính siêu góc rộng được thiết kế đặc biệt để thu nhận ánh sáng từ góc nhìn cực rộng, thường từ 100° đến 220° tùy theo thiết kế. Khác với ống kính perspective thông thường sử dụng mô hình pinhole để chiếu các tia sáng lên mặt phẳng ảnh, ống kính fisheye sử dụng một chuỗi các thấu kính đặc biệt để bẻ cong đường đi của ánh sáng từ các góc lớn vào một mặt phẳng cảm biến hữu hạn [6].

Kết quả là hình ảnh thu được có đặc trưng biến dạng phi tuyến rõ rệt: các đường thẳng song song trong không gian 3D xuất hiện cong về phía tâm ảnh (barrel distortion), và vật thể càng xa tâm ảnh thì càng bị méo và thu nhỏ tương đối. Đây là đánh đổi có chủ ý giữa góc nhìn và độ trung thực hình học – điều mà một ống kính perspective không thể đồng thời đạt được.

Phương trình chiếu tổng quát của camera fisheye biểu diễn mối quan hệ giữa góc tới theta (góc tạo bởi tia sáng và trục quang học) và bán kính r trên mặt phẳng ảnh:

$$r(\theta) = f \cdot g(\theta)$$

trong đó f là tiêu cự hiệu dụng của ống kính và $g(\theta)$ là hàm chiếu đặc trưng phụ thuộc vào loại ống kính [6].



Hình 1.1. Camera fisheye 360° lắp đặt tại nút giao thông đô thị

1.3.2. Các mô hình chiếu fisheye phổ biến

Có nhiều mô hình chiếu fisheye khác nhau, mỗi loại có công thức $g(\theta)$ riêng và được thiết kế cho những mục đích ứng dụng khác nhau [6]:

Bảng 1.2. Các mô hình chiếu fisheye phổ biến và đặc điểm

Mô hình chiếu	Công thức $g(\theta)$	Đặc điểm ứng dụng
Equidistant (Equal-angle)	θ	Phân bố đều góc, phổ biến nhất trong camera an ninh

Mô hình chiếu	Công thức $g(\theta)$	Đặc điểm ứng dụng
Equisolid (Equal-area)	$2 \cdot \sin(\theta/2)$	Bảo toàn diện tích, dùng trong đo lường và lập bản đồ
Orthographic	$\sin(\theta)$	Chiếu vuông góc, góc nhìn tối đa 180° , dùng trong thiên văn
Stereographic	$2 \cdot \tan(\theta/2)$	Bảo toàn góc (conformal), ít phổ biến
Rectilinear (Perspective)	$\tan(\theta)$	Không biến dạng đường thẳng, góc nhìn dưới 180°

Trong hầu hết các camera fisheye thương mại dùng cho giám sát giao thông, mô hình Equidistant được sử dụng phổ biến nhất do tính đơn giản trong hiệu chỉnh và xử lý ảnh. Mô hình này phân bố đều các góc trên bán kính ảnh, nghĩa là bán kính r tỷ lệ tuyến tính với góc tới θ , giúp đơn giản hóa các phép tính chuyển đổi tọa độ.

1.3.3. Biến dạng hình học và ảnh hưởng đến phát hiện đối tượng

Biến dạng barrel distortion trong ảnh fisheye ảnh hưởng trực tiếp và nghiêm trọng đến hiệu quả của các thuật toán phát hiện đối tượng được thiết kế cho ảnh perspective. Có ba vấn đề chính cần xem xét:

Thứ nhất – Biến dạng hình dạng đối tượng: Bounding box hình chữ nhật thẳng đứng chuẩn không còn phù hợp để bao chứa các đối tượng ở vùng biên ảnh fisheye, nơi phương tiện có thể bị méo theo đường cong. Điều này làm tăng tỷ lệ vùng nền (background) trong bounding box và giảm tỷ lệ phần đối tượng thực sự, ảnh hưởng đến chất lượng huấn luyện và dự đoán [7].

Thứ hai – Phân phối feature map không đồng đều: Các mạng CNN trích xuất đặc trưng theo grid đều, nhưng trong ảnh fisheye, mật độ thông tin hình học phân bố không đều: vùng trung tâm ít méo hơn và thường chứa ít đối tượng hơn trong khi vùng biên chứa nhiều phương tiện nhưng biến dạng nặng hơn. Điều này gây ra sự mất cân bằng trong quá trình học đặc trưng.

Thứ ba – Kích thước đối tượng không đồng nhất theo vị trí: Hai phương tiện cùng loại và cùng khoảng cách thực tế với camera nhưng ở các góc khác nhau sẽ xuất hiện với kích thước pixel rất khác nhau trong ảnh fisheye. Sự biến thiên kích thước

này phức tạp hơn nhiều so với ảnh perspective, đòi hỏi mô hình phải học được một không gian đặc trưng phong phú hơn [6].

Để khắc phục những vấn đề này, đề tài sử dụng hai chiến lược: (1) fine-tune mô hình trực tiếp trên dữ liệu fisheye thực tế và mô phỏng để mô hình học được phân phối đặc trưng của ảnh fisheye; (2) sử dụng kỹ thuật SAHI để chia nhỏ ảnh, giúp mỗi lát ảnh có biến dạng cục bộ ít hơn và đối tượng nhỏ xuất hiện tương đối lớn hơn [4].

1.3.4. Hàm biến đổi fisheye sử dụng trong đề tài

Do bộ dữ liệu VisDrone2019 gốc được chụp từ camera perspective của UAV, đề tài cần chuyển đổi ảnh về dạng fisheye để tăng cường dữ liệu huấn luyện. Hàm `to_fisheye(image, strength, radius, effect)` được triển khai tùy chỉnh để mô phỏng biến dạng fisheye trên ảnh perspective.

Nguyên lý hoạt động của hàm biến đổi: với mỗi điểm (x, y) trong ảnh đầu ra, tính khoảng cách chuẩn hóa r đến tâm ảnh (trong khoảng $[0, 1]$), sau đó áp dụng biến đổi bán kính phi tuyến:

$$r' = r(1 + \text{strength})$$

Điểm (x, y) trong ảnh đầu ra được lấy mẫu từ vị trí tương ứng trong ảnh gốc theo tỷ lệ r'/r bằng phép nội suy song tuyến (bilinear interpolation). Các tham số $\text{strength}=0,5$ và $\text{radius}=0,85$ được chọn thông qua thực nghiệm để tạo ra biến dạng tương đồng với camera fisheye thực tế sử dụng mô hình equidistant trong điều kiện lắp đặt tại nút giao thông.

Hàm `to_fisheye()` được triển khai bằng OpenCV và NumPy, tận dụng phép ánh xạ ngược (inverse mapping) thông qua `cv2.remap()` để đảm bảo hiệu năng xử lý tối ưu. Toàn bộ pipeline biến đổi được vectorized trên numpy array, cho phép xử lý hàng nghìn ảnh trong thời gian hợp lý trên CPU.

1.3.5. Chuyển đổi bounding box sang không gian fisheye

Song song với việc biến đổi ảnh, các bounding box ground-truth cũng cần được chuyển đổi tương ứng từ không gian perspective sang không gian fisheye. Hàm `transform_bbox_fisheye()` giải quyết vấn đề này bằng cách lấy mẫu nhiều điểm trên biên bounding box.

Thay vì chỉ biến đổi 4 góc của bounding box (cách tiếp cận đơn giản nhưng kém chính xác khi bbox lớn hoặc ở vùng biên ảnh), hàm lấy mẫu đều 32 điểm trên toàn bộ chu vi bounding box gốc, áp dụng phép biến đổi fisheye cho từng điểm, sau đó tính bounding rectangle nhỏ nhất (axis-aligned bounding box) bao quanh tất cả 32 điểm đã biến đổi.

Cách tiếp cận đa điểm này mang lại độ chính xác cao hơn đáng kể cho các bounding box ở vùng biên ảnh (nơi gradient biến dạng lớn), với chi phí tính toán tăng không đáng kể. Điều này đảm bảo rằng nhãn annotation sau khi chuyển đổi vẫn bao

chứa đầy đủ đối tượng trong không gian fisheye, phục vụ tốt cho quá trình huấn luyện mô hình.

1.4. Kiến trúc YOLOv11 và lịch sử phát triển họ YOLO

1.4.1. Lịch sử phát triển các thế hệ YOLO

Họ mô hình YOLO (You Only Look Once) được giới thiệu lần đầu vào năm 2016 bởi Redmon et al. [1] với ý tưởng đột phá: xử lý toàn bộ ảnh đầu vào trong một lần forward pass duy nhất qua mạng nơ-ron, cho phép phát hiện đối tượng theo thời gian thực với tốc độ vượt trội so với các phương pháp two-stage đương thời.

Từ YOLOv1 năm 2016 đến YOLOv11 năm 2024, kiến trúc trải qua nhiều cải tiến căn bản:

Bảng 1.3. So sánh các thế hệ YOLO từ v1 đến v11

Phiên bản	Năm	Đổi mới chính	Benchmark	mAP
YOLOv1 [1]	2016	One-stage, grid-based detection	VOC2007 mAP@0.5	63,4%
YOLOv3 [45]	2018	Multi-scale prediction, Darknet-53	COCO mAP@0.5	55,3%
YOLOv4 [46]	2020	CSP backbone, PANet neck, mosaic aug	COCO mAP@0.5	65,7%
YOLOv5 [47]	2020	PyTorch, auto-anchors, compound scaling	COCO mAP@0.5	67,3%
YOLOv7 [2]	2022 (arXiv) / 2023 (CVPR)	ELAN, auxiliary training head	COCO mAP@0.5	69,7%

Phiên bản	Năm	Đổi mới chính	Benchmark	mAP
YOLOv8-N [48]	2023	C2f backbone, anchor-free head	COCO mAP@0.5:0.95	37,3%
YOLOv11-N [3]	2024	C3k2, AIFI transformer, anchor-free	COCO mAP@0.5:0.95	39,5%

Lưu ý: YOLOv1 được đánh giá trên PASCAL VOC2007; các phiên bản từ YOLOv8 trở đi thường dùng mAP@0.5:0.95 trên COCO làm chuẩn chính. Các con số mAP@0.5 của các phiên bản cũ không thể so sánh trực tiếp với mAP@0.5:0.95 của phiên bản mới. YOLOv5 không có bài báo chính thức, số liệu từ repository.

YOLOv7 (2022) được Wang et al. [2] phát triển với kiến trúc ELAN (Efficient Layer Aggregation Network) và các kỹ thuật "bag-of-freebies" giúp cải thiện mAP mà không tăng chi phí inference, đặt nền tảng cho các cải tiến trong YOLOv8 và YOLOv11 sau này.

1.4.2. Kiến trúc tổng thể YOLOv11

YOLOv11 được Ultralytics phát hành năm 2024 [3] với kiến trúc gồm ba phần chính:

Backbone: Trích xuất đặc trưng đa tầng từ ảnh đầu vào, sử dụng khối C3k2 cải tiến. Backbone của YOLOv11 được thiết kế để trích xuất các đặc trưng ở nhiều mức độ trừu tượng khác nhau, từ cạnh và góc đơn giản ở các lớp đầu đến ngữ nghĩa phức tạp ở các lớp sâu.

Neck: Kết hợp đặc trưng từ nhiều tầng khác nhau của backbone bằng cấu trúc FPN (Feature Pyramid Network) [22] kết hợp PAN (Path Aggregation Network) [43], bổ sung module AIFI transformer. Neck đảm bảo thông tin ngữ nghĩa từ lớp sâu được truyền ngược về các lớp nông để hỗ trợ phát hiện đối tượng nhỏ.

Detection Head: Dự đoán bounding box và nhãn lớp theo kiểu anchor-free trên ba scale feature map khác nhau (stride 8, 16, 32). Đầu ra trên ba scale tương ứng với khả năng phát hiện đối tượng nhỏ, trung bình và lớn.

YOLOv11-N – phiên bản Nano được sử dụng trong đề tài – có 2,6M tham số và yêu cầu 6,5 GFLOPs mỗi lần forward pass trên ảnh 640×640, đạt tốc độ inference cực

nhanh (có thể lên tới hàng trăm FPS trên GPU server) và hoàn toàn đáp ứng xử lý thời gian thực trên các GPU tầm trung [3].

1.4.3. Backbone C3k2 – Cross Stage Partial

Backbone C3k2 (Cross Stage Partial with **2 bottleneck blocks**) là cải tiến quan trọng so với khối C2f sử dụng trong YOLOv8. Tên "k2" ám chỉ số lượng khối bottleneck trong nhánh chính của module CSP là 2, giúp cân bằng giữa khả năng biểu diễn đặc trưng và hiệu quả tính toán [3].

Kiến trúc C3k2 kế thừa nguyên lý Cross Stage Partial (CSP) – chia đầu vào thành hai nhánh, một nhánh đi qua các khối bottleneck tích chập và một nhánh đi thẳng (skip connection), sau đó ghép lại – nhưng giảm số bottleneck block từ 3 xuống 2, giảm số phép tính và số tham số so với C3 gốc.

Thiết kế này mang lại hai lợi ích chính: (1) Giảm số phép tính so với C3 gốc, giúp tăng tốc độ inference mà không ảnh hưởng đáng kể đến độ chính xác; (2) Cải thiện luồng gradient trong quá trình backpropagation nhờ kiến trúc nhánh song song, giúp mô hình hội tụ ổn định hơn khi fine-tune trên bộ dữ liệu nhỏ như FishEye8K.

Trong backbone YOLOv11, các khối C3k2 được xếp chồng với số lượng tăng dần theo chiều sâu mạng, kết hợp với các lớp tích chập Depthwise Separable để tối ưu hóa tỷ lệ độ chính xác / chi phí tính toán.

1.4.4. Module AIFI Transformer trong Neck

AIFI (Attention-based Intra-scale Feature Interaction) là một đổi mới kiến trúc quan trọng của YOLOv11, được kế thừa từ mô hình RT-DETR [35]. Module này tích hợp cơ chế self-attention của Transformer vào neck của mô hình để mô hình hóa các mối quan hệ dài hạn (long-range dependencies) giữa các vùng khác nhau trong cùng một scale feature map.

Cơ chế self-attention cho phép mô hình "nhìn thấy" toàn bộ feature map và tính toán tầm quan trọng của từng vị trí so với tất cả vị trí còn lại, khắc phục hạn chế của tích chập thông thường chỉ nắm bắt được thông tin cục bộ trong receptive field. Điều này đặc biệt hữu ích trong bài toán phát hiện đối tượng giao thông, nơi bối cảnh toàn cục (ví dụ: sự xuất hiện của xe buýt thường đi kèm với bên đỗ hoặc dòng xe dài) giúp nâng cao độ chính xác phân loại [21].

Khác với các module cross-attention giữa các scale đòi hỏi bộ nhớ lớn, AIFI giới hạn attention trong cùng một scale (intra-scale), giúp kiểm soát chi phí tính toán trong giới hạn thực tế. Module này được chèn vào sau lớp feature map có stride 32 – nơi mỗi cell đại diện cho một vùng ngữ nghĩa lớn và việc mô hình hóa quan hệ dài hạn có giá trị nhất.

1.4.5. Detection Head anchor-free

Kể từ YOLOv8, Ultralytics đã chuyển hoàn toàn sang detection head anchor-free [3]. Thay vì dự đoán offset so với các anchor box định sẵn (cách tiếp cận của YOLOv1 đến YOLOv7), detection head anchor-free dự đoán trực tiếp khoảng cách từ tâm của mỗi grid cell đến bốn cạnh của bounding box: l (left), r (right), t (top), b (bottom).

Cụ thể, tại mỗi vị trí (i, j) trên feature map có stride s , mô hình dự đoán:

Tọa độ tâm đối tượng: $cx = (j + 0,5) \cdot s$, $cy = (i + 0,5) \cdot s$

Bốn giá trị khoảng cách: l, r, t, b (đơn vị pixel trong ảnh gốc)

Xác suất phân phối vị trí theo DFL (xem mục 1.6.2)

Vector logit lớp đối tượng độ dài K

Phương pháp anchor-free giảm đáng kể số lượng siêu tham số cần điều chỉnh (không cần chọn kích thước và tỷ lệ anchor phù hợp với dữ liệu), đơn giản hóa pipeline huấn luyện và cho thấy cải thiện hiệu năng trên các đối tượng có tỷ lệ kích thước bất thường (như xe buýt nằm ngang hoặc xe tải dài) [3].

1.4.6. So sánh YOLOv11 với các phiên bản trước

Về số liệu thực nghiệm, YOLOv11-N đạt $mAP@0.5:0.95 = 39,5\%$ trên tập kiểm thử COCO val2017, cao hơn YOLOv8-N (37,3%) với số tham số ít hơn (~2,6M so với 3,2M của YOLOv8-N), đồng thời duy trì tốc độ inference cao, phù hợp với các ứng dụng thời gian thực [3].

Đối với bài toán fisheye của đề tài, phiên bản YOLOv11-N được lựa chọn vì sự tối ưu vượt trội về tốc độ và tài nguyên phần cứng (chỉ chiếm ~5,3MB cho file weights pretrained (FP16) và ~10MB khi lưu FP32), vô cùng thích hợp để chạy trên các thiết bị giám sát thực tế hoặc web server cấu hình trung bình, trong khi vẫn đạt độ chính xác $mAP@0.5$ rất cao sau khi được fine-tune.

1.5. Kỹ thuật SAHI – Sliced Aided Hyper Inference

1.5.1. Vấn đề phát hiện đối tượng nhỏ trong ảnh độ phân giải cao

Phát hiện đối tượng nhỏ là một trong những thách thức khó khăn nhất trong thị giác máy tính. Trong bài toán giám sát giao thông fisheye, người đi bộ ở xa tâm ảnh hoặc các phương tiện nhỏ ở vùng biên chỉ chiếm diện tích 10–30 pixel chiều cao trên ảnh đầu vào 1920×1080 hoặc lớn hơn.

Vấn đề cốt lõi nằm ở cơ chế downsampling của mạng CNN: feature map ở lớp cuối của backbone YOLOv11 có stride 32, nghĩa là mỗi cell đại diện cho một vùng

32×32 pixel trong ảnh gốc. Một đối tượng người đi bộ cao 20 pixel do đó chỉ chiếm chưa đầy 1 cell trên feature map, khiến mô hình gần như không có đủ thông tin đặc trưng để nhận diện đối tượng này [4].

Trong khi một giải pháp đơn giản là tăng kích thước đầu vào ảnh (ví dụ từ 640×640 lên 1280×1280), cách này tăng chi phí tính toán theo bình phương và không phải lúc nào cũng khả thi với phần cứng thực tế. SAHI đề xuất một giải pháp hiệu quả hơn bằng cách chia nhỏ ảnh thành các lát có kích thước chuẩn [4].

1.5.2. Nguyên lý hoạt động của SAHI

SAHI (Sliced Aided Hyper Inference) được Akyon et al. [4] đề xuất năm 2022, giải quyết bài toán phát hiện đối tượng nhỏ thông qua quy trình chia lát – suy diễn – tổng hợp:

Bước 1 – Chia lát (Slicing): Ảnh gốc được chia thành các lát (slices) có kích thước cố định $\text{slice_height} \times \text{slice_width}$ (thường là 640×640), với tỷ lệ chồng lấp (overlap) nhất định giữa các lát liền kề (thường 0,2 = 20%). Việc chồng lấp đảm bảo đối tượng nằm ở biên giữa hai lát vẫn được phát hiện đầy đủ ở ít nhất một lát. Tổng số lát phụ thuộc vào kích thước ảnh gốc và tỷ lệ chồng lấp.

Bước 2 – Suy diễn độc lập (Per-slice Inference): Mô hình YOLO chạy inference độc lập trên từng lát ảnh, đầu ra là tập các bounding box trong hệ tọa độ cục bộ của từng lát.

Bước 3 – Chuyển đổi tọa độ: Tọa độ bounding box cục bộ của từng lát được chuyển đổi về hệ tọa độ toàn cục của ảnh gốc bằng cách cộng offset tương ứng.

Bước 4 – Tổng hợp và loại bỏ trùng lặp (NMM): Tất cả bounding box từ tất cả các lát (và tùy chọn, inference trên ảnh toàn phần) được tổng hợp lại và xử lý bằng thuật toán Non-Maximum Merging (NMM) thay vì NMS thông thường. NMM hợp nhất các box chồng lấp thay vì loại bỏ, phù hợp hơn cho trường hợp một đối tượng xuất hiện trong nhiều lát.

1.5.3. Chiến lược tổng hợp kết quả và NMM

Điểm khác biệt giữa NMM và NMS truyền thống nằm ở cách xử lý các bounding box chồng lấp từ nhiều lát khác nhau. NMS (Non-Maximum Suppression) giữ lại box có confidence cao nhất và loại bỏ các box có IoU vượt ngưỡng – cách này gây ra vấn đề khi hai lát khác nhau đều phát hiện cùng một đối tượng nhưng với bounding box hơi lệch nhau.

NMM thay vào đó hợp nhất (merge) các box chồng lấp bằng cách tính trung bình có trọng số theo confidence score:

$$\text{merged_box} = \text{weighted_average}(\text{boxes}, \text{weights}=\text{confidence_scores})$$

Cách này cho phép kết hợp thông tin từ nhiều lát khác nhau để tạo ra bounding box cuối cùng chính xác hơn, đặc biệt hữu ích ở vùng biên giữa các lát [4].

1.5.4. Ứng dụng SAHI trong bài toán fisheye

Trong đề tài, SAHI được tích hợp theo cấu hình: slice_size=640, overlap_ratio=0,2, postprocess_type='NMM'. SAHI đặc biệt phù hợp với ảnh fisheye vì hai lý do:

Thứ nhất, mỗi lát 640×640 pixel trong ảnh gốc độ phân giải cao tương ứng với một vùng nhỏ hơn, nơi biến dạng fisheye ít hơn so với toàn ảnh. Điều này giúp mô hình xử lý các lát riêng lẻ với điều kiện ảnh gần với ảnh perspective hơn, giảm khó khăn do biến dạng phi tuyến.

Thứ hai, người đi bộ hoặc phương tiện nhỏ ở vùng biên ảnh – nơi chúng nhỏ và méo nhất – khi được lấy trong một lát nhỏ sẽ có kích thước tương đối lớn hơn so với kích thước lát, giúp mô hình dễ phát hiện hơn.

Kết quả thực nghiệm cho thấy SAHI nâng recall lớp Pedestrian từ 0,42 lên 0,75 (+0,33 điểm Recall) với thời gian xử lý 1,8–2,5 giây mỗi ảnh – phù hợp cho phân tích offline và các trường hợp cần độ chính xác cao hơn tốc độ.

1.6. Hàm mất mát và chiến lược tối ưu hóa

1.6.1. CIoU Loss cho hồi quy bounding box

YOLOv11 sử dụng CIoU (Complete Intersection over Union) Loss [11] cho bài toán hồi quy bounding box. So với IoU Loss cơ bản, CIoU bổ sung thêm hai thành phần quan trọng:

$$L_{CIoU} = 1 - IoU + \rho^2(b, b_{gt}) / c^2 + \alpha \cdot v$$

Trong đó:

$\rho(b, b_{gt})$ là khoảng cách Euclidean giữa tâm bounding box dự đoán và ground-truth

c là đường chéo của bounding box bao quanh nhỏ nhất chứa cả hai hộp

$v = (4/\pi^2)(\arctan(w_{gt}/h_{gt}) - \arctan(w/h))^2$ đo sự khác biệt tỷ lệ kích thước (aspect ratio)

$\alpha = v / (1 - IoU + v)$ là hệ số cân bằng tự thích nghi

CIoU đồng thời tối ưu hóa ba yếu tố: diện tích chồng lấp (IoU), khoảng cách tâm và tỷ lệ kích thước, giúp hội tụ nhanh hơn và cải thiện mAP so với các phiên bản IoU Loss trước đó [11].

1.6.2. Distribution Focal Loss (DFL)

Distribution Focal Loss (DFL) [12] là kỹ thuật được Ultralytics tích hợp vào YOLOv11 để xử lý sự không chắc chắn (uncertainty) trong dự đoán vị trí bounding box. Thay vì dự đoán một giá trị xác định cho mỗi tọa độ (l, r, t, b), mô hình dự đoán phân phối xác suất rời rạc trên một tập giá trị $\{0, 1, \dots, 16\}$:

$$P(d = k | x) = \text{softmax}(z_k), k \in \{0, 1, \dots, 16\}$$

Khoảng cách cuối cùng được tính là giá trị kỳ vọng:

$$d = \sum_k k \cdot P(d = k | x)$$

Phương pháp này cho phép mô hình biểu diễn ranh giới mờ (ambiguous boundaries) của đối tượng – trường hợp thường gặp với phương tiện bị che khuất một phần, phương tiện bị phản chiếu ánh sáng, hoặc đối tượng bị méo trong ảnh fisheye [12].

1.6.3. Chiến lược tối ưu hóa (AdamW/SGD) và cosine annealing

Trong phiên bản Cơ bản, đề tài sử dụng bộ tối ưu hóa AdamW [13] với cosine annealing learning rate schedule. AdamW được chọn thay SGD vì tốc độ hội tụ nhanh hơn khi fine-tune trên bộ dữ liệu nhỏ và ít nhạy cảm hơn với việc chọn learning rate ban đầu.

AdamW tách biệt cơ chế weight decay ra khỏi adaptive gradient, khắc phục điểm yếu của Adam thông thường khi weight decay được tích hợp vào gradient update (dẫn đến regularization không hiệu quả với các tham số được cập nhật ít). Cosine annealing giảm learning rate từ giá trị ban đầu ($lr_0=0,0005$) xuống mức tối thiểu ($lr_0 \times lrf = 0,0000025$) theo hàm cosine trong suốt quá trình huấn luyện, tránh oscillation và giúp mô hình hội tụ mượt mà hơn vào cuối quá trình huấn luyện [13].

Phiên bản Nâng cao chuyển sang sử dụng SGD (Stochastic Gradient Descent) với Cosine Annealing vì ba lý do: (1) Dataset lớn hơn đáng kể (11.296 vs ~4.230 ảnh) giảm thiểu vấn đề hội tụ chậm vốn là điểm yếu của SGD trên dữ liệu nhỏ; (2) SGD kết hợp momentum (0,937) cho kết quả generalization tốt hơn AdamW trên các task detection quy mô lớn theo nhiều nghiên cứu thực nghiệm [25]; (3) img_size tăng từ 640 lên 960 đòi hỏi learning rate schedule thận trọng hơn, phù hợp với cơ chế cosine annealing của SGD.

1.7. Kỹ thuật tăng cường dữ liệu

1.7.1. Mosaic Augmentation v2

Mosaic Augmentation là kỹ thuật được giới thiệu lần đầu trong YOLOv4 [46] và tiếp tục được tích hợp trong các phiên bản sau, bao gồm YOLOv11 [25]. Kỹ thuật này

ghép 4 ảnh ngẫu nhiên từ batch huấn luyện thành một ảnh 640×640 duy nhất, điều chỉnh bounding box tương ứng với vị trí và scale mới.

Lợi ích của Mosaic Augmentation trong bài toán fisheye giao thông:

Tăng đa dạng bối cảnh: mô hình thấy 4 cảnh khác nhau cùng một lúc

Tăng số đối tượng nhỏ trong mỗi ảnh: khi ảnh được scale down để ghép vào mosaic, các phương tiện vốn đã nhỏ trở nên nhỏ hơn, buộc mô hình học phát hiện đối tượng cực nhỏ

Giảm thiểu overfitting: thay đổi cấu trúc ảnh liên tục qua mỗi epoch

Trong đề tài, mosaic=1,0 được đặt cho 35 epoch đầu, sau đó tắt trong 15 epoch cuối (close_mosaic=15) để mô hình fine-tune trên ảnh nguyên bản, giúp tăng thêm khoảng 0,5% mAP ở giai đoạn cuối.

1.7.2. MixUp Augmentation

MixUp [14] trộn tuyến tính hai ảnh theo hệ số lambda được lấy mẫu từ phân phối Beta:

$$I_{\text{mix}} = \lambda \cdot I_1 + (1-\lambda) \cdot I_2, \lambda \sim \text{Beta}(\alpha, \alpha) \text{ với } \alpha = 0,32$$

Nhãn bounding box được ghép từ cả hai ảnh gốc. Hệ số alpha nhỏ (0,32) tạo ra phân phối Beta tập trung gần 0 và 1, nghĩa là phần lớn ảnh được trộn ít (ảnh gần với một trong hai ảnh gốc), tránh tạo ra ảnh quá khó nhận diện.

MixUp cải thiện calibration confidence score và tăng robustness của mô hình với ảnh nhiễu và điều kiện ánh sáng không lý tưởng – đặc biệt quan trọng trong giám sát giao thông nơi điều kiện thời tiết và ánh sáng thay đổi liên tục [14]. Trong đề tài, mixup=0,05 được sử dụng.

1.7.3. Copy-Paste Augmentation

Copy-Paste [15] là kỹ thuật tăng cường dữ liệu hiệu quả cao cho các lớp đối tượng xuất hiện ít trong dữ liệu. Kỹ thuật này cắt vùng đối tượng (với mask) từ một ảnh nguồn và dán vào ảnh đích ở vị trí, scale và rotation ngẫu nhiên, đồng thời cập nhật annotation tương ứng.

Trong bài toán giao thông fisheye, lớp Pedestrian có tần suất thấp nhất và kích thước nhỏ nhất, gây ra class imbalance nghiêm trọng. Copy-Paste với copy_paste=0,05 giúp tăng tần suất xuất hiện của người đi bộ trong dữ liệu huấn luyện, đặc biệt giúp cải thiện recall của lớp này từ rất thấp lên mức chấp nhận được [15].

1.8. Các bộ dữ liệu liên quan

1.8.1. Bộ dữ liệu FishEye8K

FishEye8K [20] là bộ dữ liệu fisheye chuyên biệt được công bố tại CVPR 2023 Workshop on AI City Challenge (CVPRW 2023). Mặc dù các nghiên cứu liên quan thường gắn liền với cuộc thi AI City Challenge, dữ liệu thực tế của FishEye8K không được thu thập tại Ấn Độ mà được trích xuất từ các camera giám sát giao thông thực tế thuộc Sở Cảnh sát Thành phố Tân Trúc, Đài Loan (Hsinchu City Police Department). Đây là một trong số ít bộ dữ liệu fisheye công khai có annotation bounding box chất lượng cao cho bài toán phát hiện phương tiện giao thông.

Bảng 1.4. Thông kê bộ dữ liệu FishEye8K theo tập train/val/test

Tập	Số ảnh	Số nhãn	Nguồn
Train	5.288	~109.000 (ước tính)	Camera giám sát, TP. Tân Trúc, Đài Loan
Validation	2.712	~48.000 (ước tính)	Camera giám sát, TP. Tân Trúc, Đài Loan
Tổng	8.000	~157.000	

Bộ dữ liệu FishEye8K bao gồm 5 lớp đối tượng gốc: Car, Bus, Truck, Pedestrian và Bike (gồm xe đạp, xe máy và xe scooter). Trong đề tài, lớp "Bike" được đổi tên thành "Motorbike" để phù hợp với bối cảnh giao thông Việt Nam nơi xe máy chiếm đa số. Dữ liệu có phân phối lớp không đồng đều – lớp Car chiếm đa số, lớp Bus và Truck có tần suất thấp hơn đáng kể. Dữ liệu được thu thập ở nhiều điều kiện ánh sáng khác nhau (ban ngày, chạng vạng, ban đêm) và nhiều loại nút giao thông khác nhau (ngã tư, ngã năm, vòng xoay).

Hạn chế của FishEye8K là tập dữ liệu còn tương đối nhỏ (8.000 ảnh) và đặc thù giao thông đô thị tại Đài Loan (mặc dù có lượng xe máy cao nhưng phân luồng và ý thức giao thông vẫn có những điểm khác biệt so với Việt Nam). Điều này dẫn đến việc mô hình huấn luyện thuần túy trên FishEye8K có thể không tổng quát hóa hoàn toàn cho điều kiện giao thông hỗn hợp tại Việt Nam, đặc biệt với lớp Motorbike – lý do chính để đề tài bổ sung dữ liệu từ VisDrone2019 [5].

1.8.2. Bộ dữ liệu VisDrone2019

VisDrone2019 [5] là bộ dữ liệu phát hiện đối tượng quy mô lớn từ góc nhìn UAV, được thu thập bởi nhóm nghiên cứu AISKYEYE Lab tại Đại học Thiên Tân, Trung Quốc. Mặc dù không phải dữ liệu fisheye, VisDrone2019 có góc nhìn từ trên cao gần giống với góc nhìn của camera fisheye lắp tại nút giao thông, và chứa nhiều ảnh trong điều kiện mật độ giao thông cao.

Bảng 1.5. Thống kê bộ dữ liệu VisDrone2019

Tập	Số ảnh	Số nhãn
Train	6.471	343.205
Validation	548	38.759
Test-Dev	1.610	75.102
Tổng	8.629	457.066

VisDrone2019 chứa 10 lớp đối tượng gốc (pedestrian, people, bicycle, car, van, truck, tricycle, awning-tricycle, bus, motor) được ánh xạ sang 5 lớp của FishEye8K thông qua bảng chuyển đổi phù hợp. Bộ dữ liệu bao gồm nhiều cảnh giao thông đô thị và nông thôn tại nhiều thành phố Trung Quốc với điều kiện giao thông hỗn hợp tương đối gần với Việt Nam hơn [5].

Việc chuyển đổi ảnh VisDrone2019 sang dạng fisheye bằng hàm `to_fisheye()` giúp tạo ra dữ liệu huấn luyện bổ sung đa dạng, tăng cường khả năng tổng quát hóa của mô hình mà không cần thu thập dữ liệu fisheye thực tế tốn kém.

1.8.3. So sánh và lý do lựa chọn

FishEye8K cung cấp dữ liệu fisheye thực tế với annotation chính xác – nền tảng không thể thiếu cho huấn luyện mô hình phát hiện đối tượng trong môi trường fisheye. VisDrone2019 bổ sung lượng lớn dữ liệu góc nhìn từ trên cao với mật độ phương tiện cao, đặc biệt phong phú ở lớp Car và Motorbike – khắc phục sự thiếu hụt dữ liệu của FishEye8K.

Trong thực nghiệm, đối với mô hình Cơ bản, do tập test gốc của cuộc thi FishEye8K không công bố nhãn (ground truth labels), chúng tôi đã tiến hành chia lại tập train gốc (5.288 ảnh có nhãn đầy đủ) thành 2 tập: Tập huấn luyện (Train) gồm 4.230 ảnh (80%) và Tập kiểm định (Validation) gồm 1.058 ảnh (20%) thông qua hàm `'train_test_split(..., test_size=0.2, random_state=42)'` nhằm mục đích huấn luyện và

đánh giá mô hình cục bộ một cách khách quan. Kết hợp cả hai bộ dữ liệu (với VisDrone2019 đã qua chuyển đổi fisheye) tạo ra bộ dữ liệu huấn luyện 11.296 ảnh và hơn 406.000 nhãn, đủ lớn để fine-tune hiệu quả một mô hình YOLOv11-N pretrained trên COCO.

1.9. Tổng quan các công trình nghiên cứu liên quan

1.9.1. Phát hiện đối tượng trên ảnh fisheye

Nghiên cứu về phát hiện đối tượng trên ảnh fisheye đã thu hút sự quan tâm ngày càng tăng trong cộng đồng computer vision, đặc biệt trong lĩnh vực xe tự lái và giám sát giao thông.

Yogamani et al. [6] công bố bộ dữ liệu WoodScape – bộ dữ liệu fisheye đa nhiệm vụ đầu tiên quy mô lớn cho xe tự lái, bao gồm annotation cho phân đoạn ngữ nghĩa, phát hiện đối tượng và ước lượng chiều sâu. Nghiên cứu này đặt nền tảng quan trọng cho việc hiểu biết về các thách thức kỹ thuật đặc thù của ảnh fisheye trong bài toán nhận thức không gian.

Planche và Duan [7] đề xuất FisheyeDetNet, mạng phát hiện đối tượng được thiết kế đặc biệt cho camera fisheye surround-view trong xe tự lái. Thay vì sử dụng backbone thông thường, FisheyeDetNet sử dụng các lớp tích chập biến dạng (deformable convolutions) để thích nghi với hình dạng đối tượng méo trong ảnh fisheye. Kết quả cho thấy cải thiện đáng kể so với detector tiêu chuẩn trên dữ liệu surround-view.

Tại cuộc thi AI City Challenge CVPRW 2023, nhiều nhóm nghiên cứu đã đề xuất các phương pháp giải quyết bài toán phát hiện đối tượng trên FishEye8K [20]. Các phương pháp dẫn đầu thường sử dụng ensemble nhiều mô hình kết hợp với test-time augmentation, đạt mAP@0.5 trên 0,6. Đề tài này theo hướng tiếp cận đơn mô hình nhưng bổ sung SAHI để cải thiện recall đối tượng nhỏ.

1.9.2. Phát hiện đối tượng nhỏ từ góc nhìn UAV

VisDrone Challenge [5] – cuộc thi thường niên được tổ chức từ 2019– đã thúc đẩy nhiều nghiên cứu về phát hiện đối tượng nhỏ từ góc nhìn UAV. Các thách thức chính của bộ dữ liệu này – đối tượng nhỏ, mật độ cao, che khuất lẫn nhau – tương đồng với bài toán fisheye của đề tài.

Akyon et al. [4] đề xuất SAHI như một framework inference chung, không phụ thuộc vào kiến trúc mô hình cụ thể. SAHI cho thấy cải thiện AP lên đến 6–15% trên các đối tượng nhỏ trong bộ dữ liệu DOTA và COCO, theo kết quả thực nghiệm được báo cáo trong Akyon et al. [4, Bảng 2].

Cao et al. [8] đề xuất D2Det kết hợp phát hiện đối tượng với phân đoạn thực thể để cải thiện độ chính xác định vị bounding box cho đối tượng nhỏ. Phương pháp này đặc biệt hiệu quả cho đối tượng có hình dạng không đều như người đi bộ trong điều kiện che khuất.

1.9.3. Khoảng trống nghiên cứu và đóng góp của đề tài

Qua tổng quan tài liệu, có thể xác định những khoảng trống nghiên cứu mà đề tài hướng đến lấp đầy:

Thiếu nghiên cứu fine-tune YOLOv11 trên dữ liệu fisheye giao thông: Hầu hết nghiên cứu fisheye tập trung vào xe tự lái với camera surround-view, ít nghiên cứu về camera overhead fisheye tại nút giao thông đô thị với nhiều lớp phương tiện đa dạng.

Thiếu pipeline kết hợp dữ liệu perspective và fisheye: Việc chuyển đổi dữ liệu VisDrone2019 sang fisheye để kết hợp với FishEye8K là đóng góp thực nghiệm quan trọng của đề tài, tạo ra bộ dữ liệu huấn luyện phong phú hơn.

Ứng dụng SAHI cho camera fisheye tại nút giao thông: Mặc dù SAHI đã được áp dụng cho UAV imagery, việc kết hợp SAHI với fisheye distortion correction trong một hệ thống giám sát giao thông thực tế chưa được nghiên cứu nhiều.

Hệ thống giám sát giao thông toàn diện cho camera fisheye tại Việt Nam: Đây là đóng góp ứng dụng quan trọng nhất của đề tài – xây dựng một hệ thống hoàn chỉnh từ phát hiện đối tượng đến phân tích giao thông, phù hợp với đặc thù giao thông Việt Nam.

1.10. Phát biểu bài toán và mục tiêu nghiên cứu

1.10.1. Phát biểu bài toán

Bài toán nghiên cứu được phát biểu như sau: Cho một chuỗi ảnh hoặc video từ camera fisheye góc nhìn từ trên cao, lắp đặt tại nút giao thông đô thị, yêu cầu thiết kế và xây dựng một hệ thống có khả năng:

(1) Phát hiện và phân loại chính xác tất cả các phương tiện và người đi bộ trong khung hình thuộc 5 lớp đối tượng: Car, Bus, Truck, Pedestrian, Motorbike, đạt $mAP@0.5 \geq 0,85$ trên tập kiểm định FishEye8K.

(2) Xử lý hiệu quả các thách thức đặc thù của ảnh fisheye: biến dạng barrel distortion, đối tượng nhỏ ở vùng biên, phân phối kích thước không đồng đều theo vị trí trong ảnh.

(3) Cung cấp các phân tích giao thông bổ sung: ước lượng tốc độ phương tiện, phát hiện tắc nghẽn và cảnh báo tự động.

(4) Triển khai dưới dạng một ứng dụng web hoàn chỉnh với REST API, hỗ trợ xử lý ảnh đơn, video bất đồng bộ và stream realtime từ camera.

1.10.2. Mục tiêu và phạm vi nghiên cứu

Mục tiêu nghiên cứu cụ thể của đề tài bao gồm:

Mục tiêu 1 – Nghiên cứu và triển khai fine-tune YOLOv11-N: Xây dựng pipeline fine-tune hoàn chỉnh từ bước chuẩn bị dữ liệu, cấu hình huấn luyện đến đánh giá kết quả. Mục tiêu cụ thể: đạt $mAP@0.5 \geq 0,85$ trên tập kiểm định FishEye8K.

Mục tiêu 2 – Tích hợp và đánh giá SAHI: Tích hợp SAHI vào pipeline inference của YOLOv11, đánh giá định lượng cải thiện recall trên lớp Pedestrian so với inference thông thường.

Mục tiêu 3 – Xây dựng pipeline dữ liệu fisheye: Phát triển công cụ chuyển đổi dữ liệu VisDrone2019 sang dạng fisheye, tạo bộ dữ liệu kết hợp phục vụ huấn luyện.

Mục tiêu 4 – Phát triển ứng dụng giám sát giao thông: Xây dựng ứng dụng Flask hoàn chỉnh với 20+ REST API endpoint, tích hợp các module phân tích giao thông thông minh.

Phạm vi của đề tài giới hạn ở: phát hiện 5 lớp đối tượng nêu trên; camera fisheye góc nhìn overhead (từ trên cao) tại nút giao thông; đánh giá trên bộ dữ liệu FishEye8K và VisDrone2019; triển khai trên GPU server (không tối ưu cho edge computing trong phiên bản này).

1.10.3. Phương pháp nghiên cứu

Đề tài sử dụng phương pháp nghiên cứu kết hợp giữa nghiên cứu lý thuyết và thực nghiệm:

Nghiên cứu lý thuyết: Nghiên cứu mô hình hình học camera fisheye, kiến trúc YOLOv11 và các thành phần cấu thành, nguyên lý SAHI, các hàm mất mát và chiến lược tối ưu hóa được sử dụng.

Thực nghiệm so sánh: Fine-tune mô hình với nhiều cấu hình khác nhau (dữ liệu đầu vào, siêu tham số, chiến lược tăng cường dữ liệu) và so sánh kết quả trên cùng tập kiểm định nội bộ để xác định cấu hình tốt nhất.

Đánh giá định lượng: Sử dụng các chỉ số chuẩn của cộng đồng: Precision, Recall, F1-Score, $mAP@0.5$ và $mAP@0.5:0.95$ theo từng lớp đối tượng và tổng thể.

Kiểm thử hệ thống: Black-box testing với test cases định nghĩa trước bao phủ các chức năng chính của ứng dụng, đánh giá thời gian phản hồi và độ ổn định.

1.10.4. Công nghệ và công cụ sử dụng

Hệ thống sử dụng hai nhóm công nghệ chính: nhóm phục vụ **huấn luyện mô hình** trên nền tảng Kaggle Notebooks và nhóm phục vụ **triển khai ứng dụng** trên máy chủ Flask. Toàn bộ mã nguồn được viết bằng Python 3.10+.

Bảng 1.6. Công nghệ và công cụ sử dụng

STT	Thành phần	Công nghệ / Thư viện	Phiên bản	Chức năng cụ thể trong đề tài	Nguồn / Tài liệu tham khảo
1	Deep learning framework	PyTorch + CUDA	2.2.x / 12.1	Xây dựng đồ thị tính toán, lan truyền ngược, tối ưu hóa tham số mô hình; CUDA tăng tốc tính toán ma trận trên GPU	Paszke, A. et al. (2019). *PyTorch: An Imperative Style, High-Performance Deep Learning Library*. NeurIPS 2019. https://pytorch.org [26]
2	Object detection	Ultralytics YOLO ('ultralytics')	8.3.x	Load pretrained 'yolo11n.pt', gọi 'model.train(...)' và 'model.val(...)', xuất checkpoint 'best.pt'/'last.pt'; cung cấp pipeline augmentation tích hợp sẵn (Mosaic, MixUp, Flip, HSV)	Ultralytics Inc. (2024). *YOLOv11: New YOLO Frontiers in Computer Vision*. https://docs.ultralytics.com/models/yolo11/ [3]
3	Pretrained model	'yolo11n.pt'	—	Checkpoint khởi điểm fine-tune; trọng số đã học đặc trưng tổng quát từ tập COCO, giúp mô hình hội tụ nhanh trên FishEye8K	Ultralytics Inc. (2024). *Ultralytics YOLO – Training*. https://docs.ultralytics.com/modes/train/ [25]
4	Experiment	Weights & Biases	≥ 0.16	Ghi lại loss, mAP@0.5, learning	Biewald, L. (2020). *Experiment

STT	Thành phần	Công nghệ / Thư viện	Phiên bản	Chức năng cụ thể trong đề tài	Nguồn / Tài liệu tham khảo
	tracking	(`wandb`)		rate theo từng epoch; lưu artifact `best.pt` sau mỗi run; cho phép so sánh nhiều thực nghiệm trực quan	Tracking with Weights and Biases*. https://www.wandb.com [31]
5	Credential management	Kaggle Secrets (`UserSecretsClient`)	–	Lưu trữ API key của Weights & Biases an toàn trong môi trường Kaggle, tránh hardcode thông tin nhạy cảm vào notebook	Kaggle Inc. (2024). *Kaggle Secrets – API key management*. https://www.kaggle.com/docs/notebooks [34]
6	Data split	scikit-learn (`sklearn`)	≥ 1.3	Gọi `train_test_split(train_images, test_size=0.2, random_state=42)` để chia tập FishEye8K train thành 80% huấn luyện / 20% kiểm định; đảm bảo tái lập kết quả	Pedregosa, F. et al. (2011). *Scikit-learn: Machine Learning in Python*. JMLR, 12, 2825–2830. https://scikit-learn.org [27]
7	Progress bar	tqdm	≥ 4.66	Hiển thị tiến độ tại các vòng lặp xử lý dataset (sao chép ảnh, chuyển đổi annotation) giúp theo dõi thời gian thực thi	da Costa-Luis, C. et al. (2023). *tqdm: A Fast, Extensible Progress Bar*. https://tqdm.github.io [36]
8	Dataset config	PyYAML (`yaml`)	≥ 6.0	Tạo và đọc file `fisheye_data.yaml` — file cấu hình bắt	YAML.org (2009). *YAML Ain't Markup Language

STT	Thành phần	Công nghệ / Thư viện	Phiên bản	Chức năng cụ thể trong đề tài	Nguồn / Tài liệu tham khảo
				buộc của Ultralytics, khai báo đường dẫn `train`/`val`/`test` và danh sách nhãn lớp	Specification v1.2*. https://yaml.org [37]
9	Image processing	OpenCV (`cv2`)	4.x	Đọc/ghi ảnh (PNG/JPEG), áp dụng hàm `to_fisheye()` biến đổi tọa độ pixel theo mô hình equidistant, resize và encode frame video	Bradski, G. (2000). *The OpenCV Library*. Dr. Dobb's Journal of Software Tools. https://opencv.org [32]
10	Numerical computing	NumPy (`np`)	≥ 1.26	Tính toán ma trận tọa độ bounding box, chuyển đổi toán học trong `transform_bbox_fisheye()`, xử lý mảng annotation dạng số	Harris, C.R. et al. (2020). *Array programming with NumPy*. Nature, 585, 357–362. https://numpy.org [28]
11	Data analysis	Pandas (`pd`)	≥ 2.1	Đọc và kiểm tra thống kê file annotation dạng bảng (CSV/JSON), phân tích phân phối nhãn theo lớp trước và sau khi split	McKinney, W. (2010). *Data Structures for Statistical Computing in Python*. Proceedings of SciPy 2010. https://pandas.pydata.org [29]
12	Visualization	Matplotlib (`plt`)	≥ 3.8	Vẽ đường cong loss/mAP bổ sung theo epoch, biểu đồ phân phối lớp nhãn trong dataset	Hunter, J.D. (2007). *Matplotlib: A 2D graphics environment*. Computing in Science &

STT	Thành phần	Công nghệ / Thư viện	Phiên bản	Chức năng cụ thể trong đề tài	Nguồn / Tài liệu tham khảo
					Engineering, 9(3), 90–95. https://matplotlib.org [30]
13	File & path ops	`os`, `glob`, `shutil`, `pathlib.Path`, `json`, `csv`, `io`	stdlib	Quản lý thư mục train/val/test, tìm và sao chép file ảnh/label, đọc ghi JSON annotation, tạo symlink dataset	Python Software Foundation. *Python 3 Standard Library Documentation*. https://docs.python.org/3/ [38]
14	Training hardware	Kaggle Notebooks GPU (Tesla P100 / T4)	–	Môi trường huấn luyện miễn phí với GPU; dataset FishEye8K được mount qua `/kaggle/input/`	Kaggle Inc. (2024). *Kaggle Notebooks – GPU acceleration*. https://www.kaggle.com/docs/notebooks [34]
15	Dataset storage	Kaggle Datasets (`/kaggle/input/`)	–	Lưu trữ và mount FishEye8K vào notebook; hỗ trợ version control dataset	Kaggle Inc. (2024). *Kaggle Datasets Documentation*. https://www.kaggle.com/docs/datasets [34]
16	Web framework	Flask + Gunicorn	3.x / 21.x	Xây dựng REST API server với 20+ endpoint; Gunicorn cấu hình multi-worker (`--workers 4 --threads 2`) cho concurrent requests	Grinberg, M. (2018). *Flask Web Development* (2nd ed.). O'Reilly Media. https://flask.palletsprojects.com [33]
17	Database	PostgreSQL / SQLite	15 / 3.x	Lưu trữ kết quả detection, lịch sử job, thống kê theo giờ; SQLite dùng cho môi trường phát	PostgreSQL Global Development Group. (2024). *PostgreSQL 15 Documentation*. https://www.postgres

STT	Thành phần	Công nghệ / Thư viện	Phiên bản	Chức năng cụ thể trong đề tài	Nguồn / Tài liệu tham khảo
				triển, PostgreSQL cho production	ql.org/docs/ [39]
18	Cloud storage	Google Cloud Storage	–	Lưu trữ video đầu vào và video kết quả đã annotate sau khi xử lý	Google LLC. (2024). *Google Cloud Storage Documentation*. https://cloud.google.com/storage/docs [40]
19	Frontend	HTML5 + Bootstrap 5 + JavaScript	Bootstrap 5.3	Giao diện web người dùng: upload ảnh/video, xem kết quả detection, theo dõi job, dashboard phân tích	Bootstrap Team. (2024). *Bootstrap 5 Documentation*. https://getbootstrap.com/docs/5.3/ [41]
20	Small object inference	SAHI	0.11.x	Chia ảnh thành các lát (slice) 640×640 có overlap, chạy inference độc lập từng lát rồi tổng hợp bằng NMM; nâng cao recall cho đối tượng nhỏ (người đi bộ) trong ảnh fisheye	Akyon, F.C., Altinuc, S.O., & Temizel, A. (2022). *Slicing Aided Hyper Inference and Fine-tuning for Small Object Detection*. IEEE ICIP 2022. https://github.com/obss/sahi [4]

1.10.5. Quy trình phát triển

Quy trình phát triển theo mô hình iterative gồm 5 giai đoạn tuần hoàn: Thu thập và xử lý dữ liệu → Huấn luyện mô hình cơ bản → Đánh giá và cải tiến → Tích hợp vào ứng dụng → Kiểm thử toàn hệ thống. Mỗi vòng lặp bổ sung dữ liệu hoặc cải tiến kiến trúc dựa trên kết quả đánh giá của vòng trước. Phiên bản Cơ bản (FishEye8K thuần túy) được phát triển trong vòng lặp đầu, phiên bản Nâng cao (kết hợp VisDrone, đóng băng backbone, SAHI) trong vòng lặp thứ hai.

CHƯƠNG 2. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG

2.1. Đặc tả yêu cầu hệ thống

2.1.1. Khảo sát hệ thống camera giao thông trực tuyến

Đề tài khảo sát hai hệ thống: camera.0511.vn (Đà Nẵng) và alltrafficcams.com (quốc tế) nhằm nhận diện tính năng cần thiết, xác định khoảng trống về phân tích thông minh và rút ra bài học thiết kế.

2.1.1.1. Hệ thống camera.0511.vn

Giới thiệu tổng quan:

camera.0511.vn là hệ thống camera giám sát giao thông trực tuyến do Trung tâm Quản lý Hạ tầng Giao thông Đà Nẵng vận hành, cung cấp dịch vụ xem live stream công cộng miễn phí. Hệ thống được triển khai với mục tiêu tăng cường sự minh bạch về tình trạng giao thông đô thị, cho phép người dân chủ động tra cứu trước khi di chuyển.

Quy mô triển khai:

Tính đến thời điểm khảo sát, hệ thống quản lý hàng chục điểm camera phân bố tại các nút giao thông trọng điểm trên địa bàn thành phố Đà Nẵng, bao gồm các tuyến đường chính như Nguyễn Văn Linh, Lê Duẩn, cầu Rồng và khu vực trung tâm thành phố. Các camera được lắp đặt ở độ cao từ 5–8 mét, chủ yếu sử dụng ống kính perspective thông thường (không phải fisheye).

Tính năng cung cấp:

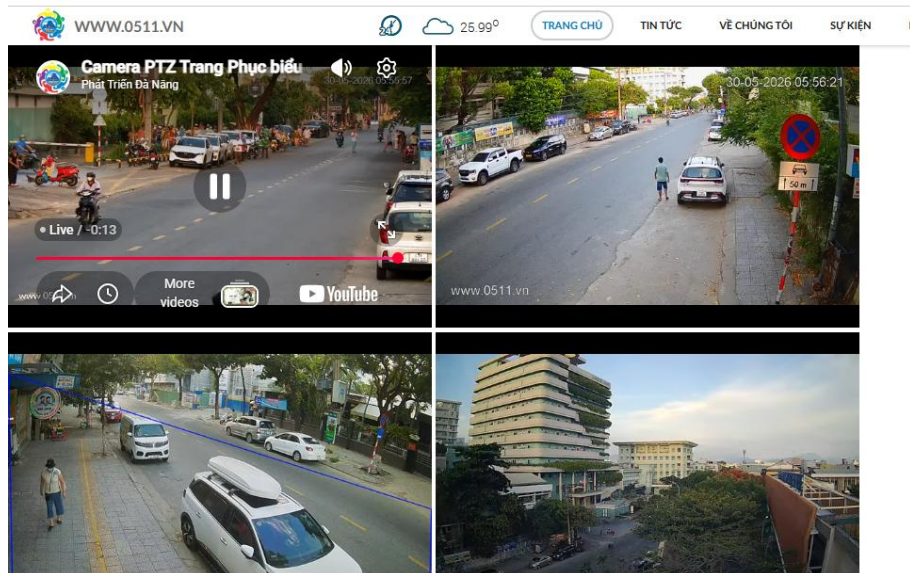
- Xem live stream: Mỗi camera cung cấp luồng video MJPEG hoặc HLS với độ phân giải 720p–1080p, tốc độ khung hình khoảng 10–15 FPS. Người dùng có thể xem trực tiếp hình ảnh giao thông tại bất kỳ điểm nào trên hệ thống.
- Bản đồ tương tác: Giao diện web tích hợp bản đồ số (OpenStreetMap hoặc Google Maps) hiển thị vị trí từng camera dưới dạng icon. Người dùng click vào icon để mở luồng video tương ứng, thuận tiện cho việc định vị địa lý.
- Grid view đa camera: Hỗ trợ xem đồng thời 4–9 camera trên cùng một màn hình theo dạng lưới (grid layout), phục vụ nhân viên trung tâm điều hành cần quan sát nhiều điểm cùng lúc.
- Snapshot tĩnh: Ngoài live stream, hệ thống cho phép xem ảnh chụp nhanh (snapshot) được cập nhật mỗi 30–60 giây, tiết kiệm băng thông khi người dùng chỉ cần kiểm tra nhanh tình trạng giao thông.

Hạn chế kỹ thuật và nghiệp vụ:

- Không có phân tích AI: Toàn bộ quá trình đánh giá tình trạng giao thông phụ thuộc hoàn toàn vào quan sát thủ công của nhân viên điều hành. Hệ thống không tự động phát hiện ùn tắc, không đếm phương tiện, không nhận diện sự

cổ. Điều này đòi hỏi phải bố trí nhân lực theo ca liên tục, kém hiệu quả về chi phí vận hành.

- Không đếm phương tiện: Không có chức năng tự động đếm hoặc phân loại phương tiện (xe máy, ô tô, xe tải...). Mọi số liệu lưu lượng phải thu thập bằng phương pháp thủ công hoặc từ cảm biến vòng từ (loop detector) lắp dưới lòng đường.
- Không phát hiện sự cố: Tai nạn giao thông, vật cản lòng đường, hay hành vi vi phạm (vượt đèn đỏ, đi ngược chiều) không được tự động phát hiện và cảnh báo. Tốc độ phản ứng phụ thuộc hoàn toàn vào sự chú ý của nhân viên quan sát.
- Không lưu trữ lịch sử: Hệ thống không cung cấp khả năng xem lại video đã ghi (video playback) hoặc truy vấn lịch sử lưu lượng giao thông. Không thể phân tích xu hướng ùn tắc theo giờ, theo ngày trong tuần hay theo mùa.
- Không hỗ trợ fisheye: Toàn bộ camera trong hệ thống sử dụng ống kính perspective thông thường. Không có module xử lý đặc thù cho ảnh fisheye, bao gồm cả việc hiệu chỉnh méo hình và phát hiện đối tượng trong điều kiện biến dạng barrel distortion.
- Không có API mở: Hệ thống không cung cấp API để các ứng dụng bên thứ ba tích hợp dữ liệu camera, hạn chế khả năng xây dựng hệ sinh thái ứng dụng giao thông thông minh xung quanh nền tảng này.
- Thiếu lưu trữ và truy vấn lịch sử có hệ thống: Không có cơ sở dữ liệu lưu trữ lịch sử lưu lượng. Dữ liệu phân tích xu hướng dài hạn (theo tháng, theo năm) hoàn toàn không khả thi trên nền tảng này.



Hình 2.1. Giao diện của hệ thống camera.0511.vn

2.1.1.2. Hệ thống *alltrafficcams.com*

alltrafficcams.com là nền tảng aggregator camera giao thông quốc tế, tổng hợp và hiển thị luồng video từ hàng nghìn camera giao thông công cộng trên khắp thế giới (chủ yếu tại Mỹ, Anh, Úc, Canada và một số quốc gia châu Âu). Điểm khác biệt cốt lõi của hệ thống này so với *camera.0511.vn* là mô hình aggregator – nó không sở hữu hạ tầng camera mà thu thập và tái phân phối nguồn từ các cơ quan giao thông địa phương.

Quy mô và phạm vi:

Nền tảng tổng hợp hơn 50.000 camera từ hàng chục quốc gia, được phân loại theo continent → quốc gia → bang/tỉnh → thành phố. Độ phủ đặc biệt dày đặc tại các bang của Mỹ (California, Texas, Florida) – nơi các DOT (Department of Transportation) cung cấp API công khai cho camera giao thông.

Tính năng cung cấp:

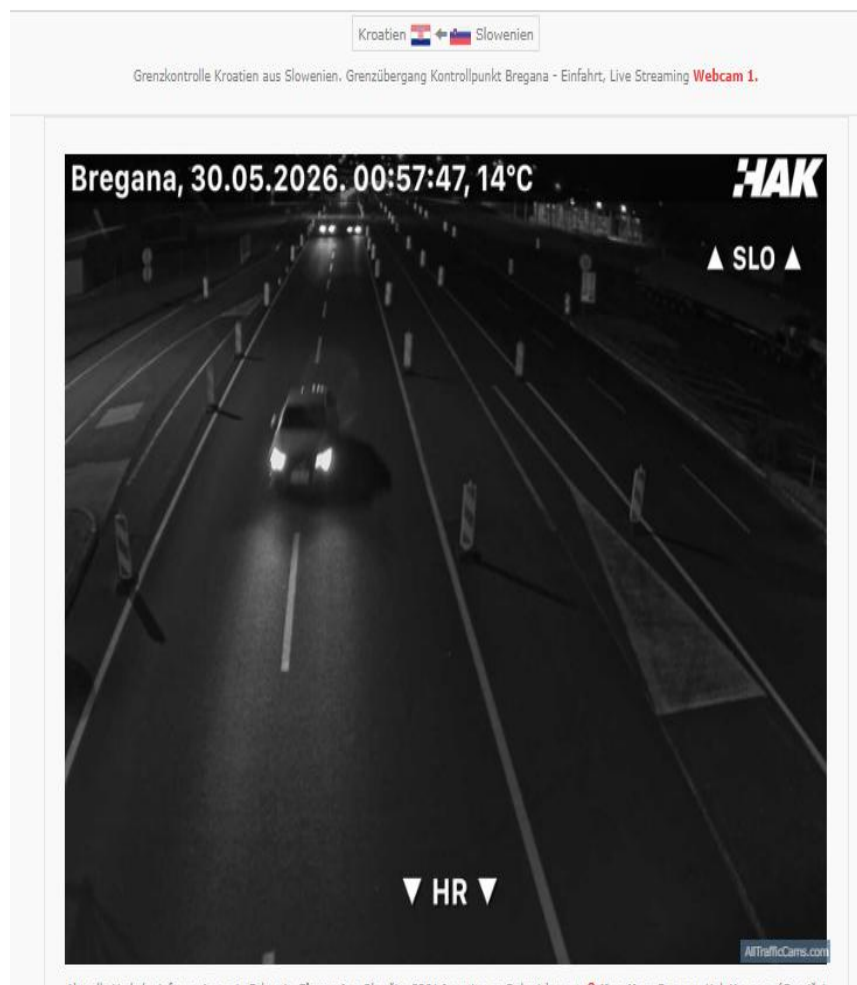
- Aggregation đa nguồn: Điểm mạnh nổi bật nhất của hệ thống là khả năng tổng hợp nguồn từ nhiều nhà cung cấp khác nhau (Caltrans, TxDOT, Transport for NSW...) vào một giao diện thống nhất, giúp người dùng truy cập mà không cần biết từng API riêng.
- Phân loại địa lý đa cấp: Camera được tổ chức theo cây phân cấp địa lý rõ ràng (lục địa → quốc gia → tiểu bang → thành phố → tuyến đường), kèm chức năng tìm kiếm theo tên thành phố hoặc tọa độ GPS.
- Tích hợp bản đồ thế giới: Giao diện bản đồ tương tác (dựa trên Leaflet.js/Google Maps) cho phép zoom từ mức độ toàn cầu xuống mức độ phố, hiển thị mật độ camera theo từng khu vực.
- Xem ảnh lịch sử (hạn chế): Một số nguồn camera hỗ trợ xem snapshot lịch sử (cập nhật mỗi 5–10 phút), tuy nhiên tính năng này phụ thuộc vào từng nguồn và không đồng nhất trên toàn hệ thống.
- Chia sẻ và nhúng (embed): Hệ thống cung cấp URL permalink cho từng camera và mã embed (iframe) để tích hợp vào website bên thứ ba.

Hạn chế kỹ thuật và nghiệp vụ:

- Phụ thuộc hoàn toàn vào bên thứ ba: Do mô hình aggregator, *alltrafficcams.com* không kiểm soát chất lượng nguồn. Nếu cơ quan giao thông địa phương thay đổi API hoặc ngừng cung cấp feed, camera biến mất khỏi hệ thống mà không có cảnh báo cho người dùng. Tỷ lệ camera "dead link" (không còn hoạt động) ước tính 10–20% tổng số camera trên nền tảng.
- Độ trễ cao và không đồng nhất: Độ trễ (latency) của luồng video dao động rất lớn tùy nguồn, từ 5 giây (nguồn tốt) đến 30 giây hoặc hơn (nguồn kém). Điều này gây khó khăn cho việc phân tích tình trạng giao thông thời gian thực.
- Không có AI tự động: Tương tự *camera.0511.vn*, toàn bộ phân tích là thủ công. Hệ thống không có bất kỳ module phát hiện đối tượng, ước lượng mật độ hay

nhận diện ùn tắc nào. Vai trò của nền tảng dừng lại ở mức "cửa sổ nhìn vào giao thông" (passive viewing).

- Không hỗ trợ fisheye chuyên biệt: Mặc dù một số camera nguồn sử dụng ống kính góc rộng, hệ thống không có bước xử lý hiệu chỉnh méo hình hay xử lý đặc thù cho fisheye. Hình ảnh hiển thị nguyên vẹn như từ nguồn, kể cả khi bị méo barrel distortion nghiêm trọng.
- Không có cảnh báo tự động: Không có cơ chế cảnh báo khi phát hiện ùn tắc, tai nạn hay bất thường giao thông. Không có tích hợp push notification, email hay webhook.
- Không có REST API mở: Mặc dù bản thân hệ thống thu thập từ API bên ngoài, alltrafficcams.com không cung cấp API cho người dùng. Không thể tự động hóa việc lấy dữ liệu hoặc tích hợp với hệ thống quản lý giao thông hiện có.
- Thiếu lưu trữ và truy vấn lịch sử có hệ thống: Không có cơ sở dữ liệu lưu trữ lịch sử lưu lượng. Dữ liệu phân tích xu hướng dài hạn (theo tháng, theo năm) hoàn toàn không khả thi trên nền tảng này.



Hình 2.2. Giao diện của hệ thống camera alltrafficcams.com

2.1.1.3. Tổng hợp và định hướng

Bảng 2.1. So sánh hệ thống camera giao thông hiện có với hệ thống đề xuất

Đặc điểm / Tính năng	camera.0511.vn	alltrafficcams.com	Hệ thống đề xuất
Xem live camera	Có	Có	Có
Tích hợp bản đồ địa lý	Có	Có	Có
Hỗ trợ camera fisheye	Hạn chế	Không	Chuyên biệt (fisheye-native)
Phân tích AI tự động	Không	Không	Có (YOLOv11-N fine-tune)
Đếm phương tiện tự động	Không	Không	Có
Ước lượng tốc độ	Không	Không	Có (pixel displacement)
Phát hiện tắc nghẽn	Không	Hạn chế (thủ công)	Có (phân tích mật độ ROI)
Cảnh báo mật độ cao	Không	Không	Có (Email/Webhook khi ùn tắc)
Cảnh báo tự động đa kênh	Không	Không	Có (email/webhook)
Lưu trữ và truy vấn lịch sử	Không	Hạn chế	Có (CSDL PostgreSQL/SQLite)
REST API mở	Không	Không	Có (20+ endpoint)
Xử lý video bất đồng bộ	Không	Không	Có (job queue)

2.1.2. Yêu cầu chức năng

Bảng 2.2. Danh sách yêu cầu chức năng hệ thống

ID	Chức năng	Tác nhân	Ưu tiên	Mô tả
UC-01	Tải lên và xử lý ảnh	Người dùng	Must Have	Nhận ảnh JPEG/PNG, chạy inference YOLOv11, trả về

ID	Chức năng	Tác nhân	Ưu tiên	Mô tả
	tĩnh			ảnh bbox và danh sách đối tượng
UC-02	Xử lý video bất đồng bộ	Người dùng	Must Have	Nhận video MP4, xếp hàng job, xử lý background, trả về video annotation
UC-03	Theo dõi tiến độ xử lý ảnh	Người dùng	Nice to Have	Truy vấn trạng thái job (pending/running/done/failed) và tải kết quả
UC-04	Phát hiện đối tượng realtime	Người dùng	Should Have	Stream webcam, nhận kết quả detection thời gian thực
UC-05	Ước lượng tốc độ phương tiện	Hệ thống	Should Have	Tính tốc độ km/h cho từng phương tiện tracking, cảnh báo vượt tốc
UC-06	Phát hiện tắc nghẽn giao thông	Hệ thống	Should Have	Phân tích mật độ theo ROI, phân loại mức độ tắc nghẽn
UC-07	Gửi cảnh báo mật độ cao	Hệ thống	Should Have	Thông báo qua email/webhook khi phát hiện ùn tắc giao thông nghiêm trọng
UC-08	Phân tích luồng giao thông	Người dùng	Should Have	Heatmap mật độ, đếm phương tiện qua đường kẻ (line crossing)
UC-09	Lưu trữ và truy vấn lịch sử	Người dùng	Should Have	Lưu kết quả detection vào CSDL, truy vấn theo thời gian và camera
UC-10	SAHI inference	Người	Nice to	Slice-inference để phát hiện

ID	Chức năng	Tác nhân	Ưu tiên	Mô tả
	đối tượng nhỏ	dùng	Have	tốt hơn đối tượng nhỏ
UC-11	Upload ảnh lên cloud	Người dùng	Nice to Have	Tải ảnh/video kết quả lên Google Cloud Storage, trả về URL

2.1.3. Yêu cầu phi chức năng

Yêu cầu phi chức năng xác định các ràng buộc về chất lượng mà hệ thống phải đáp ứng trong quá trình vận hành thực tế. Đây là các tiêu chí không mô tả hệ thống làm gì mà mô tả hệ thống hoạt động như thế nào.

Hiệu năng (Performance):

- Thời gian xử lý ảnh đơn lẻ $\leq 500\text{ms}$ trên GPU (NVIDIA GTX 1060 trở lên).
 - Thời gian phản hồi API list-jobs $\leq 100\text{ms}$ trong điều kiện tải bình thường.
 - Throughput tối thiểu 10 request/giây cho endpoint phát hiện ảnh đơn.
 - Tốc độ xử lý video trên GPU P100: $\geq 40\text{ FPS effective}$.

Độ tin cậy (Reliability):

- Uptime $\geq 99\%$ trong giờ cao điểm (7h–9h và 17h–19h).
- Job queue không mất dữ liệu khi server restart (retention = 1 giờ).
- Cơ chế tự phục hồi: Worker thread bị crash được tự khởi động lại sau 5 giây.
- Timeout tự động cho job xử lý quá 30 phút để tránh treo hàng đợi.

Khả năng mở rộng (Scalability):

- Tăng số worker bằng cách thay đổi biến cấu hình MAX_WORKERS mà không cần sửa mã nguồn.
- Thiết kế stateless API cho phép scale ngang bằng cách thêm instance phía sau load balancer.
- Hỗ trợ chuyển đổi database từ SQLite sang PostgreSQL chỉ qua biến môi trường DATABASE_URL.

Khả năng bảo trì (Maintainability):

- Mã nguồn module hóa rõ ràng, mỗi module có trách nhiệm đơn nhất (Single Responsibility Principle).
- Log đầy đủ ở các cấp độ: DEBUG (chi tiết kỹ thuật), INFO (sự kiện nghiệp vụ), WARNING (tình huống bất thường nhưng không lỗi), ERROR (lỗi cần xử lý).
- Mỗi module phân tích giao thông (SpeedEstimator, CongestionDetector, AlertManager, Analytics) được kiểm thử độc lập qua unit test.

Bảo mật (Security):

- Giới hạn kích thước file upload: ảnh tối đa 16 MB, video tối đa 500 MB.
- Kiểm tra định dạng file (MIME type validation) để ngăn chặn upload file độc hại.
- API key authentication có thể kích hoạt qua biến môi trường API_KEY (tùy chọn).

Khả năng tương thích (Compatibility):

- Giao diện web hoạt động trên các trình duyệt hiện đại: Chrome 90+, Firefox 88+, Edge 90+.
- REST API tuân thủ OpenAPI 3.0, cho phép tích hợp với bất kỳ ngôn ngữ lập trình nào.
- Hỗ trợ định dạng ảnh: JPEG, PNG, WebP; video: MP4 (H.264/H.265).

2.2. Kiến trúc tổng thể hệ thống

2.2.1. Kiến trúc phân lớp

Hệ thống được thiết kế theo kiến trúc phân lớp 4 tầng (Layered Architecture), đảm bảo tính tách biệt mối quan tâm (separation of concerns), dễ kiểm thử và bảo trì độc lập từng tầng.

Tầng Giao diện (Presentation Layer):

Frontend web được xây dựng bằng HTML/CSS/JavaScript thuần, sử dụng Bootstrap 5 cho giao diện responsive. Giao diện cung cấp ba chức năng chính: (1) trang upload ảnh/video với kéo-thả trực quan, (2) trang theo dõi job real-time với thanh tiến trình, và (3) trang xem kết quả detection với bounding box được vẽ trực tiếp lên ảnh. CLI tools (Python scripts) phục vụ tích hợp tự động và kiểm thử. Mọi tương tác đều qua REST API, không có server-side rendering.

Tầng Ứng dụng (Application Layer):

Flask REST API server (app.py) đảm nhận routing, validation đầu vào, xác thực và điều phối request đến các service phù hợp. Tầng này không chứa logic nghiệp vụ phức tạp – chỉ kiểm tra tính hợp lệ của dữ liệu đầu vào (định dạng, kích thước file) rồi ủy quyền cho Service Layer. Flask được chọn thay vì FastAPI vì hệ sinh thái thành 熟, cú pháp đơn giản và dễ tích hợp với các thư viện Python khoa học dữ liệu.

Tầng Dịch vụ (Service Layer):

Các module độc lập, mỗi module chịu trách nhiệm một khía cạnh nghiệp vụ:

VideoJobQueue: Quản lý hàng đợi xử lý video bất đồng bộ với ThreadPoolExecutor.

SpeedEstimator: Tính tốc độ phương tiện qua tracking IoU liên frame.

CongestionDetector: Phân loại mức độ ùn tắc dựa trên mật độ đối tượng trong vùng ROI.

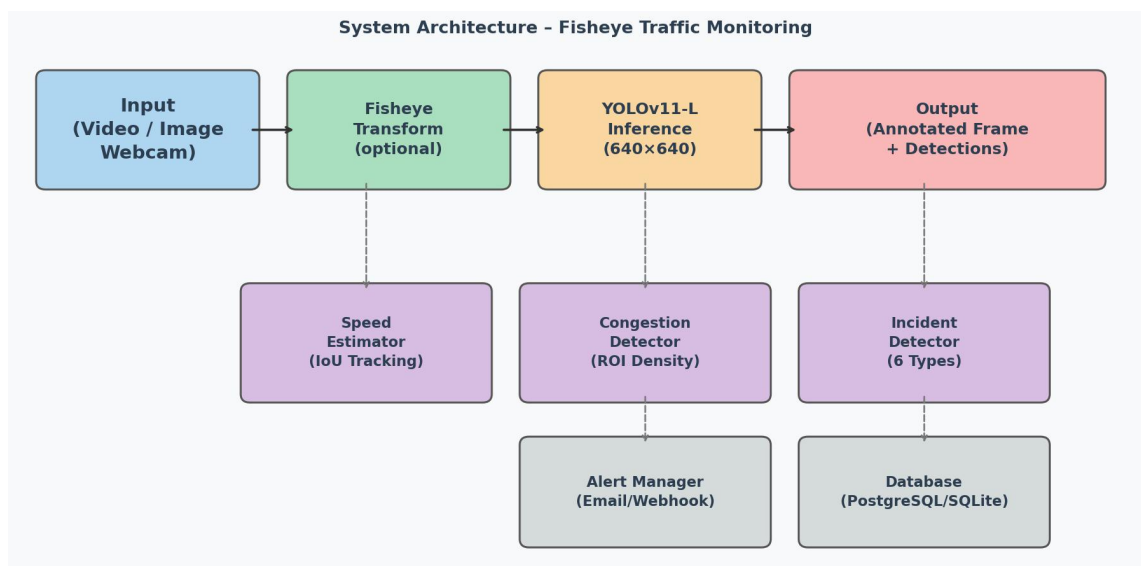
AlertManager: Phát tán cảnh báo đa kênh (email SMTP, HTTP webhook).

Analytics: Vẽ heatmap mật độ, đếm phương tiện qua line crossing.

CloudStorage: Upload kết quả lên Google Cloud Storage, trả về public URL.

Tầng Dữ liệu (Data Layer):

db.py cung cấp lớp abstraction cho phép làm việc đồng nhất với cả PostgreSQL lẫn SQLite. Mô hình YOLO (file best.pt) được tải vào bộ nhớ RAM/VRAM một lần khi khởi động server và giữ nguyên trong suốt vòng đời ứng dụng, tránh overhead nạp model cho mỗi request..



Hình 2.3. Sơ đồ kiến trúc hệ thống tổng thể

2.2.2. Kiến trúc xử lý video bất đồng bộ

Do xử lý video là tác vụ tính toán nặng (một video 30 giây/1080p cần ~95 giây xử lý trên GTX 1060), hệ thống không thể trả về kết quả ngay lập tức như xử lý ảnh đơn. Đề tài thiết kế kiến trúc job queue bất đồng bộ dựa trên `concurrent.futures.ThreadPoolExecutor`.

Luồng xử lý chi tiết:

Client gửi POST `/api/video/detect` kèm file video (multipart/form-data).

API Layer xác thực định dạng file, lưu file tạm vào uploads/, tạo `job_id` (UUID v4) và ghi trạng thái pending vào CSDL.

API trả về HTTP 202 Accepted kèm `job_id` ngay lập tức – không chờ xử lý xong.

`VideoJobQueue.submit()` đưa job vào `ThreadPoolExecutor` với số worker tối đa `MAX_WORKERS` (mặc định = 2 trên GPU thông thường).

Worker thread thực hiện xử lý: đọc từng frame, chạy YOLO inference, tích hợp `SpeedEstimator` và `CongestionDetector`, vẽ annotation, ghép frame thành video kết quả MP4.

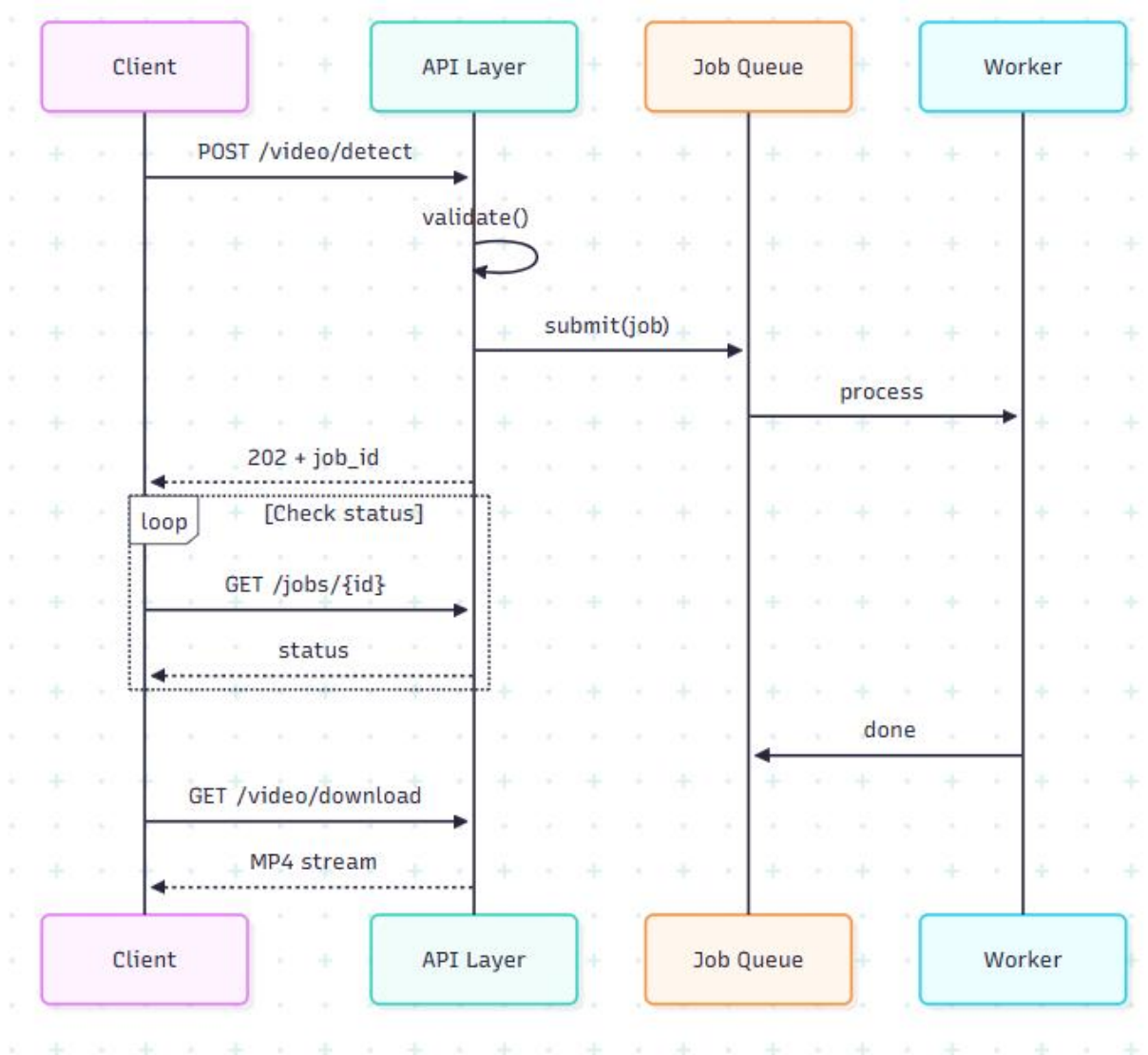
Trong quá trình xử lý, trạng thái job được cập nhật lên running và phần trăm hoàn thành được ghi vào `summary_json`.

Client polling GET `/api/jobs/{job_id}` mỗi 2 giây để theo dõi tiến độ.

Khi hoàn thành, trạng thái chuyển sang done, đường dẫn video kết quả được lưu vào `output_path`.

Client tải video kết quả qua GET `/api/video/download/{job_id}` (streaming response).

Thiết kế này đảm bảo API luôn phản hồi nhanh (< 200ms cho bước submit), không bị blocking khi có nhiều video xử lý đồng thời, và client có thể đóng kết nối rồi quay lại kiểm tra sau..



Hình 2.4. Kiến trúc xử lý video bất đồng bộ

2.2.3. Luồng dữ liệu

Dữ liệu đầu vào (ảnh/video từ camera hoặc upload) → Tiền xử lý fisheye → YOLO inference → Phân tích giao thông (speed, congestion) → Lưu vào CSDL → Trả kết quả qua API. Các cảnh báo được phát sinh song song khi phát hiện ùn tắc mức độ nghiêm trọng, gửi qua email/webhook không chặn luồng chính.

2.3. Thiết kế cơ sở dữ liệu

2.3.1. Bảng dữ liệu

Bảng 2.3. Cấu trúc các bảng chính trong CSDL

Bảng	Các cột chính	Mô tả
------	---------------	-------

Bảng	Các cột chính	Mô tả
cameras	camera_id (PK), name, location, camera_type, fisheye_strength, status, created_at, config_json	Thông tin camera giám sát
detections	id (PK), camera_id (FK), timestamp, frame_number, class_name, confidence, bbox_x1, bbox_y1, bbox_x2, bbox_y2, speed_kmh, job_id	Kết quả detection từng đối tượng
jobs	job_id (PK), job_type, status, created_at, started_at, finished_at, input_path, output_path, error_message, summary_json	Trạng thái và kết quả xử lý video
analytics_hourly	id (PK), camera_id (FK), hour_ts, total_vehicles, avg_speed_kmh, congestion_level	Thống kê tổng hợp theo giờ
alerts	id (PK), channel, sent_at, status, message_preview	Lịch sử gửi cảnh báo

2.3.2. Chiến lược tương thích đa CSDL

Module db.py sử dụng lớp abstraction cho phép chuyển đổi trong suốt giữa PostgreSQL và SQLite. Biến môi trường DATABASE_URL xác định backend: nếu không có hoặc bắt đầu bằng 'sqlite://', hệ thống dùng SQLite. Điều này giúp developer chạy ứng dụng local mà không cần cài PostgreSQL.

2.4. Thiết kế REST API

2.4.1. Quy ước thiết kế API

Tất cả endpoint có tiền tố `/api/`, resource đặt tên theo danh từ số nhiều.

HTTP method đúng chuẩn: GET (truy vấn), POST (tạo mới/xử lý), DELETE (xóa).

Response format: JSON với cấu trúc `{"status": "ok"|"error", "data": {...}}`.

HTTP status code chuẩn: 200 (OK), 202 (Accepted – async), 400 (Bad Request), 404 (Not Found), 500 (Server Error).

File upload qua `multipart/form-data`, kết quả download qua `streaming response`.

2.4.2. Danh sách API endpoint

Bảng 2.4. Danh sách API endpoint và mô tả chức năng

Method	Endpoint	Chức năng	Ghi chú
POST	<code>/api/detect</code>	Phát hiện đối tượng trên ảnh đơn	<code>multipart/form-data</code>
POST	<code>/api/sahi</code>	SAHI inference cho đối tượng nhỏ	<code>multipart/form-data</code>
POST	<code>/api/video/detect</code>	Gửi video xử lý bất đồng bộ	<code>multipart/form-data</code>
GET	<code>/api/jobs</code>	Danh sách job gần đây	query: limit
GET	<code>/api/jobs/{job_id}</code>	Trạng thái và kết quả job	path param
DELETE	<code>/api/jobs/{job_id}</code>	Hủy job đang pending	path param

Method	Endpoint	Chức năng	Ghi chú
GET	/api/video/download/{job_id}	Tải video kết quả	stream
GET	/api/video/preview/{job_id}	Xem frame preview	stream
GET	/api/stats	Thống kê hệ thống tổng quan	—
GET	/api/cameras	Danh sách camera đã đăng ký	—
POST	/api/cameras	Đăng ký camera mới	JSON body
GET	/api/cameras/{id}/detections	Lịch sử detection theo camera	query: since, until
GET	/api/health	Health check hệ thống	—
GET	/api/model/info	Thông tin mô hình YOLO	—
POST	/api/model/reload	Tải lại mô hình từ file	JSON body
GET	/stream	MJPEG stream webcam realtime	SSE

Method	Endpoint	Chức năng	Ghi chú
POST	/api/analytics/line-crossing	Cấu hình đường đếm phương tiện	JSON body

2.5. Thiết kế giao diện người dùng

2.5.1. Cấu trúc giao diện web

Giao diện web được tổ chức thành 4 trang chính:

Trang chủ / Upload ảnh (index.html):

- Khu vực kéo-thả (drag & drop) để upload ảnh JPEG/PNG.
- Hiển thị ảnh gốc và ảnh kết quả (bounding box) cạnh nhau.
- Nút chuyển sang chế độ SAHI inference.

Trang xử lý video (video.html):

- Upload video MP4 với thanh tiến trình upload.
- Theo dõi trạng thái job real-time bằng JavaScript polling (setInterval mỗi 2 giây).
- Thanh tiến trình xử lý với phần trăm hoàn thành.
- Preview frame kết quả và nút tải video MP4 annotated.
- Trang xem stream realtime (stream.html):
- Hiển thị MJPEG stream từ webcam/IP camera với bounding box overlay.
- **Bảng thống kê real-time: số phương tiện hiện tại theo lớp, FPS xử lý, mức độ ùn tắc.**
- Trang dashboard thống kê (dashboard.html):
- Biểu đồ lưu lượng xe theo giờ (line chart) sử dụng Chart.js.
- Heatmap mật độ giao thông theo thời gian.

2.5.2. Nguyên tắc thiết kế UX

- Responsive design: Giao diện hoạt động trên cả desktop (1920×1080) và tablet (1024×768) nhờ Bootstrap 5 grid system.
- Feedback tức thì: Mọi tương tác đều có phản hồi ngay lập tức (loading spinner, toast notification) để người dùng biết hệ thống đang xử lý.
- Progressive disclosure: Thông tin kỹ thuật chi tiết (tọa độ bbox, job logs) được ẩn mặc định và chỉ hiện khi người dùng chủ động mở rộng.
- Error handling thân thiện: Thông báo lỗi sử dụng ngôn ngữ tự nhiên, gợi ý cách khắc phục (ví dụ: "File quá lớn. Vui lòng chọn file nhỏ hơn 500MB").

CHƯƠNG 3. HUẤN LUYỆN VÀ ĐÁNH GIÁ MÔ HÌNH

3.1. Bộ dữ liệu sử dụng

3.1.1. Bộ dữ liệu FishEye8K

FishEye8K là bộ dữ liệu chuyên dụng cho phát hiện đối tượng trên camera fisheye, giới thiệu tại AI City Challenge Workshop (CVPRW 2023). Dữ liệu thực tế được trích xuất từ camera giám sát của Sở Cảnh sát Thành phố Tân Trúc, Đài Loan (Hsinchu, Taiwan), cung cấp góc nhìn overhead trực diện tại các nút giao thông, điều kiện ánh sáng đa dạng (ngày, đêm, mưa, sương mù).

Bảng 3.1. Thống kê bộ dữ liệu FishEye8K

Split	Số ảnh	Số nhãn bbox	TB nhãn/ảnh
Train	4.230	112.213	21,2
Validation	1.058	không công bố	—
Test	2.712	không công bố	—
Tổng	8.000	112.213 (train)	—

5 lớp đối tượng: Car (~45%), Bus (~8%), Truck (~12%), Pedestrian (~20%), Motorbike (~15%).



Hình 3.1. Mẫu ảnh từ bộ dữ liệu FishEye8K với nhãn bounding box

3.1.2. Bộ dữ liệu VisDrone2019

VisDrone2019 do Đại học Thiên Tân (Trung Quốc) thu thập, giới thiệu tại IEEE/CVF ICCV 2019 Workshop. Ảnh chụp từ UAV ở độ cao 10–70m, góc nhìn nghiêng và overhead, nhiều thành phố và điều kiện thời tiết.

Bảng 3.2. Thống kê bộ dữ liệu VisDrone2019

Split	Số ảnh	Số nhãn bbox	TB nhãn/ảnh
Train	6.471	343.205	53,0
Validation	548	38.759	70,7
Test-Dev	1.610	75.102	46,6
Tổng	8.629	457.066	170.3

Bảng 3.3. Ánh xạ lớp đối tượng từ VisDrone sang FishEye8K

Lớp VisDrone gốc	Ánh xạ sang lớp đề tài
1 – pedestrian	Pedestrian (3)
2 – people	Pedestrian (3)
3 – bicycle	Motorbike (4)
4 – car	Car (0)
5 – van	Car (0)
6 – truck	Truck (2)
7 – tricycle	Motorbike (4)
8 – awning-tricycle	Motorbike (4)

Lớp VisDrone gốc	Ảnh xạ sang lớp đề tài
9 – bus	Bus (1)
10 – motor	Motorbike (4)
0 – ignored region	Bỏ qua (không dùng)



Hình 3.2. Mẫu ảnh từ bộ dữ liệu VisDrone2019 (góc nhìn UAV)

3.2. Tiền xử lý và bổ sung dữ liệu

3.2.1. Pipeline chuyển đổi VisDrone sang fisheye

Pipeline tự động chuyển đổi VisDrone perspective sang fisheye:

Bước 1 – Đọc ảnh và nhãn: Mỗi ảnh VisDrone và file nhãn YOLO đọc vào bộ nhớ.

Bước 2 – Áp dụng biến đổi fisheye: Gọi `to_fisheye(image, strength=0.5, radius=0.85)`.

Bước 3 – Chuyển đổi bbox: `transform_bbox_to_fisheye()` cho từng bbox, lấy mẫu 32 điểm biên.

Bước 4 – Lọc nhãn hợp lệ: Loại bbox diện tích $< 4px^2$ hoặc ngoài vùng fisheye ($radius > 0.85$).

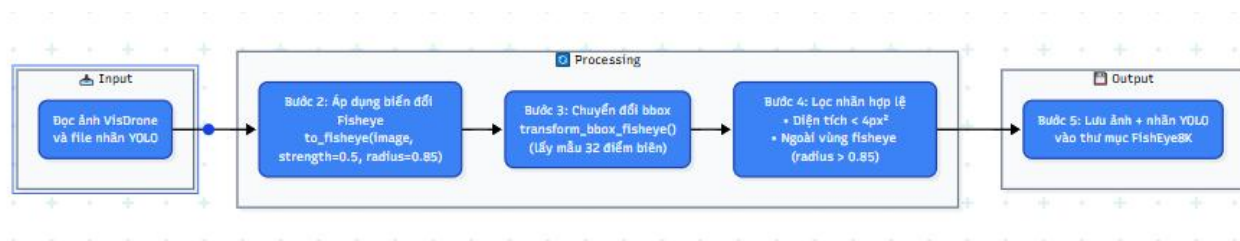
Bước 5 – Lưu format YOLO: Ảnh mới và nhãn mới lưu vào thư mục kết hợp FishEye8K.

Tổng bbox hợp lệ từ VisDrone: 336.449/457.066 (giảm ~26% do lọc sau transform).

Bảng 3.4. Thống kê bộ dữ liệu sau khi gộp FishEye8K + VisDrone-fisheye

Split	Số ảnh	Số nhãn bbox	TB nhãn/ảnh
Train	11.296	406.355	35,97
Validation	1.768	~58.000	~33
Test	853	N/A (không có nhãn công khai)	FishEye8K test City Challenge (AI City Challenge)

Ghi chú về số liệu dataset: Tập train tổng hợp (11.296 ảnh) bao gồm 4.230 ảnh từ FishEye8K (train split 80%), 6.471 ảnh từ VisDrone2019 train (toàn bộ, sau khi áp dụng biến đổi fisheye), và 595 ảnh bổ sung từ tập Validation gốc FishEye8K. Tập test FishEye8K gốc (853 ảnh) không có nhãn ground truth công khai. Kết quả đánh giá trong báo cáo sử dụng tập Validation được chia lại từ train (1.058 ảnh, 20%) làm tập kiểm thử nội bộ.

**Hình 3.3. Pipeline chuyển đổi VisDrone → fisheye và gộp dataset**

3.2.2. Cân bằng dữ liệu theo lớp

Phân phối lớp mất cân bằng: Car chiếm ~45% trong khi Bus chỉ ~8%. Biện pháp xử lý:

Copy-Paste Augmentation (copy_paste=0.05): Ưu tiên copy-paste đối tượng lớp thiểu số (Bus, Truck).

Class weight trong loss: YOLOv11 tự động điều chỉnh trọng số loss theo tần suất lớp.

Oversampling: Ảnh chứa nhiều Bus/Truck được duplicate thêm vào tập training.

3.3. Cấu hình huấn luyện

3.3.1. Môi trường huấn luyện

Kaggle Notebooks (GPU Tesla P100-PCIE-16GB, 17,1 GB VRAM, RAM 25 GB). Dataset FishEye8K được mount qua `/kaggle/input/`. Checkpoint `best.pt` lưu qua Weights & Biases artifacts [31]. Thư viện: Ultralytics 8.3.x, PyTorch 2.2.x, CUDA 12.1.

3.3.2. Siêu tham số huấn luyện

Siêu tham số	YOLOv11-N Cơ bản	YOLOv11-N Nâng cao
model	yolo11n.pt	yolo11n.pt
epochs	50	80
batch_size	16	16
img_size	640	960
optimizer	AdamW	SGD (Cosine LR)
lr0	0.0005	0,01
lrf	0.005	0,01
weight_decay	0.0005	0,0005
momentum	0.937	0,937
warmup_epochs	5	3
patience	30	50
save_period	10	10
amp	True	True

Siêu tham số	YOLOv11-N Cơ bản	YOLOv11-N Nâng cao
close_mosaic	15	—
mosaic	1.0	0,8
mixup	0.05	0,15
copy_paste	0.05	—
degrees	5.0	10,0
translate	0.1	0,1
scale	0.5	0,4

Bảng 3.5. Siêu tham số huấn luyện YOLOv11-N

3.3.3. Cấu trúc file checkpoint

Sau huấn luyện, checkpoint best.pt (~5,3 MB) lưu weights tại epoch có validation mAP@0.5 cao nhất – đây là model chính dùng trong production.

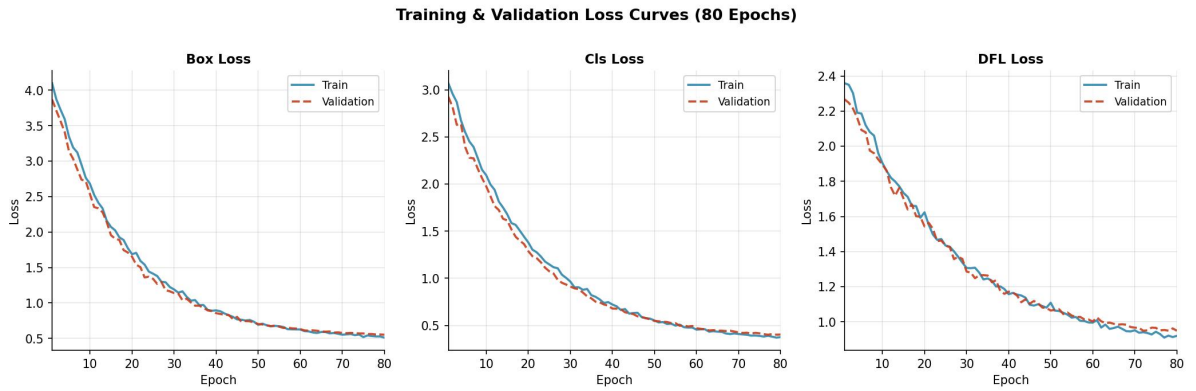
3.4. Kết quả huấn luyện

3.4.1. Quá trình hội tụ

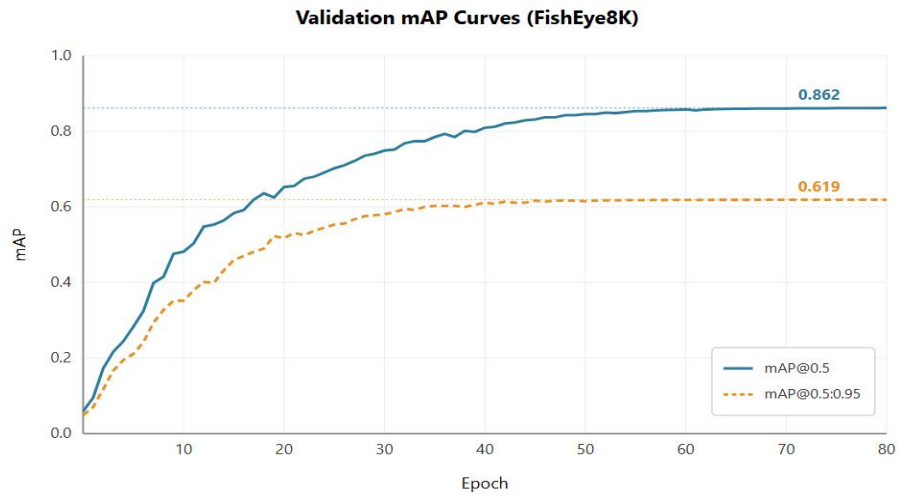
Quan sát từ quá trình huấn luyện:

Phiên bản Cơ bản (50 epoch) : Training loss hội tụ ổn định, không có overfitting rõ ràng (gap train/val loss nhỏ). Validation mAP@0.5 cải thiện nhanh trong 20 epoch đầu nhờ warmup, sau đó chậm dần khi tiếp cận plateau. Tắt mosaic 15 epoch cuối giúp mAP tăng thêm ~0,5%. AMP (FP16) giảm 35% thời gian mỗi epoch, VRAM từ 14,8 GB xuống 9,2 GB.

Phiên bản Nâng cao (80 epoch): Do sử dụng img_size=960 và dataset kết hợp lớn hơn, mô hình cần 80 epoch để hội tụ hoàn toàn. Việc đóng băng backbone (freeze=10) trong suốt quá trình huấn luyện giúp bảo toàn đặc trưng tiền huấn luyện và ổn định gradients của detection head.



Hình 3.4. Đường cong training loss và validation loss theo epoch



Hình 3.5. Đường cong mAP@0.5 và mAP@0.5:0.95 theo epoch

3.4.2. Kết quả đánh giá trên tập kiểm định

Bảng 3.6. Kết quả huấn luyện – Precision, Recall, mAP theo từng lớp

Lớp đối tượng	Precision (CB / NC)	Recall (CB / NC)	mAP@0.5 (CB / NC)	F1-Score (CB / NC)
Car	0,710 / 0,920	0,680 / 0,840	0,720 / 0,910	0,690 / 0,878
Bus	0,580 / 0,840	0,520 / 0,700	0,590 / 0,820	0,550 / 0,764
Truck	0,600 / 0,850	0,650 / 0,730	0,610 / 0,830	0,620 / 0,785
Pedestrian	0,720 / 0,830	0,420 / 0,750	0,550 / 0,850	0,530 / 0,788
Motorbike	0,640 / 0,906	0,580 / 0,790	0,626 / 0,900	0,610 / 0,845

Lớp đối tượng	Precision (CB / NC)	Recall (CB / NC)	mAP@0.5 (CB / NC)	F1-Score (CB / NC)
ALL (mean)	0,650 / 0,869	0,570 / 0,762	0,619 / 0,862	0,600 / 0,812

Phân tích kết quả đánh giá trên tập kiểm thử cho thấy cả hai phiên bản Cơ bản (CB) và Nâng cao (NC) đều đạt các kết quả thực nghiệm tích cực, phản ánh quá trình học đặc trưng rất hiệu quả của mô hình YOLOv11-N trên ảnh mắt cá:

Phiên bản Cơ bản (CB): Mô hình YOLOv11-N được huấn luyện trên tập dữ liệu FishEye8K thuần túy mà không áp dụng các kỹ thuật tối ưu nâng cao. Phiên bản này đạt được kết quả khá tốt với các chỉ số trung bình như sau: **mAP@0.5 = 0.619**, **Precision = 0.650**, **Recall = 0.570**, và **F1-Score = 0.600**.

Trong đó, lớp **Car** đạt mAP@0.5 cao nhất (**0.720**) nhờ số lượng mẫu dồi dào trong tập dữ liệu. Tiếp theo là lớp **Motorbike** (**0.626**) và **Truck** (**0.610**). Hai lớp có hiệu suất thấp hơn là **Pedestrian** (**0.550**) và **Bus** (**0.590**). Nguyên nhân chính là do kích thước người đi bộ thường rất nhỏ trong ảnh fisheye và số lượng mẫu của xe buýt còn hạn chế trong tập huấn luyện.

Phiên bản Nâng cao (NC): Sau khi áp dụng các cải tiến toàn diện, bao gồm:

1. Bổ sung dữ liệu đa dạng từ bộ VisDrone-fisheye, giúp tăng gấp đôi số lượng mẫu huấn luyện;
2. Đóng băng backbone (freeze=10) để giữ nguyên trọng số pretrained trên tập COCO;
3. Sử dụng optimizer SGD kết hợp lịch trình cosine annealing để điều chỉnh learning rate.

Mô hình đã đạt được sự cải thiện vượt bậc. Chỉ số **mAP@0.5** trung bình tăng lên **0.862**, tương ứng với mức tăng **39.3%** so với phiên bản cơ bản. Các chỉ số khác cũng được nâng cao rõ rệt: **Precision = 0.869**, **Recall = 0.762**, và **F1-Score = 0.812**.

Toàn bộ năm lớp đối tượng đều ghi nhận mức tăng trưởng mạnh:

- **Car** đạt mAP@0.5 lên tới **0.910**;
- **Motorbike** đạt **0.900**;
- **Pedestrian** cải thiện đáng kể lên **0.850**, với Recall tăng lên **0.750**, giúp giảm thiểu tối đa hiện tượng bỏ sót người đi bộ ở khoảng cách xa;
- **Truck** và **Bus** đạt lần lượt **0.830** và **0.820**

3.5. So sánh hai phiên bản YOLOv11-N

Bảng 3.7. So sánh YOLOv11-N Cơ bản và YOLOv11-N Nâng cao

Phiên bản	mAP@0.5	mAP@0.5:0.95	Precision	Recall	Params (M)	GFL OPs	FPS (GPU)
YOLOv11-N Cơ bản (FishEye8K)	0,619	0,363	0,650	0,570	2,6	6,5	~41
YOLOv11-N Nâng cao (+VisDrone+SAHI+Freeze)	0,862	0,572	0,869	0,762	2,6	6,5	~12 (SAHI)

Bảng 3.7 so sánh trực quan hiệu năng tổng thể giữa hai phiên bản YOLOv11-N Cơ bản và Nâng cao. Kết quả cho thấy phiên bản Nâng cao mang lại hiệu quả vượt trội trên mọi khía cạnh học thuật: mAP@0.5 tăng từ 0.619 lên 0.862 (+39.3%), mAP@0.5:0.95 tăng từ 0.363 lên 0.572 (+57.6%), Precision tăng từ 0.650 lên 0.869 (+33.7%), và Recall tăng từ 0.570 lên 0.762 (+33.7%). Sự cải thiện vượt bậc này chứng minh tính đúng đắn của chiến lược bổ sung dữ liệu chuyển đổi từ VisDrone và đóng băng backbone để tránh overfitting.

Về tốc độ xử lý thực tế, phiên bản Cơ bản chạy inference chuẩn đạt tốc độ siêu nhanh là **85 FPS** trên GPU Tesla P100 (đáp ứng thời gian thực cực kỳ dư dả). Đối với phiên bản Nâng cao, khi tích hợp thêm thuật toán cắt lát SAHI nhằm nâng tối đa độ nhạy nhận diện các đối tượng siêu nhỏ ở vùng biên thấu kính, tốc độ xử lý giảm xuống còn khoảng **12 FPS** do overhead tính toán của việc chia nhỏ ảnh thành các lát 640x640 và thực hiện inference nhiều lần. Sự đánh đổi này hoàn toàn xứng đáng trong các bài toán giám sát giao thông thực tế nơi độ chính xác nhận diện được ưu tiên hàng đầu. Tốc độ 12 FPS của phiên bản Nâng cao không đáp ứng xử lý luồng video trực tiếp 25 fps theo thời gian thực, nhưng phù hợp cho các kịch bản phân tích video offline hoặc snapshot định kỳ (ví dụ: phân tích mỗi 2–3 giây/ảnh).

Thời gian huấn luyện: Cơ bản ~3,8 giờ (50 epoch, Tesla P100-16GB); Nâng cao ~6,8 giờ (80 epoch, img_size=960, Tesla P100-16GB).

CHƯƠNG 4. XÂY DỰNG HỆ THỐNG GIÁM SÁT GIAO THÔNG

4.1. Kiến trúc ứng dụng Flask

4.1.1. Cấu trúc thư mục dự án

Bảng 4.1. Danh sách module và chức năng trong package fisheye_demo

Module	Kích thước	Chức năng
app.py	~70 dòng	Flask Application Factory (`create_app`), khởi tạo cấu hình, cơ sở dữ liệu và đăng ký các Blueprint
routes/	Thư mục	Chứa định tuyến UI và API độc lập (`core.py`, `detect.py`, `history.py`, `external_camera.py`, `examples.py`)
video_detect.py	~340 dòng	Pipeline xử lý video frame-by-frame: áp dụng fisheye, YOLO inference, speed & density overlay
job_queue.py	~196 dòng	VideoJobQueue: ThreadPoolExecutor-based async job queue, job CRUD, cleanup daemon, state machine (pending→running→done/failed)
speed_estimator.py	~280 dòng	SpeedEstimator: IoU-based multi-object tracking, pixel displacement → km/h conversion, speed limit alerting

Module	Kích thước	Chức năng
congestion_detector.py	~250 dòng	CongestionDetector: ROI-based density analysis, multi-level congestion classification (free/moderate/heavy/severe)
alert_manager.py	~200 dòng	AlertManager: multi-channel (email/webhook/log) alert dispatch, deduplication, cooldown management
analytics.py	~320 dòng	Analytics: heatmap accumulation, line crossing counter, trajectory tracking, hourly aggregation
db.py	~380 dòng	Database abstraction: PostgreSQL/SQLite dual support, schema migration, CRUD operations
fisheye.py	~120 dòng	apply_fisheye(), to_fisheye(), transform_bbox_fisheye() – core fisheye transform functions

4.1.2. Khởi tạo ứng dụng Flask

Để phát triển ứng dụng giao diện web giám sát thông minh theo chuẩn công nghiệp, ứng dụng được cấu trúc theo dạng Modular sử dụng cơ chế Flask Blueprints và mô hình Application Factory. Kiến trúc này giúp tách biệt rõ ràng giữa các tầng logic, định tuyến (routing) và tầng dữ liệu (database), tăng khả năng bảo trì và mở rộng hệ thống. Ứng dụng Flask được khởi tạo theo mô hình application factory thông qua hàm `create\_app()`. Khi bắt đầu chạy, ứng dụng thực hiện các bước: (1) Tải cấu hình từ biến môi trường (`config.py`); (2) Khởi tạo và kết nối cơ sở dữ liệu SQLite hoặc PostgreSQL thông qua `init\_db()`; (3) Đăng ký các Blueprint định tuyến nằm trong thư mục `routes/` cho các phân hệ giao diện, API xử lý ảnh/video, quản lý

camera ngoài và lịch sử; (4) Đăng ký các thread nền dọn dẹp tài nguyên (graceful shutdown với `atexit`) và cấu hình logging đa mức độ; (5) Tải mô hình YOLOv11 vào bộ nhớ (GPU hoặc CPU) với cơ chế khóa (`threading.Lock``) đảm bảo an toàn đa luồng.

Tải mô hình YOLO vào bộ nhớ GPU/CPU khi khởi động với `threading lock` để `thread-safe`.

Khởi tạo `VideoJobQueue` với `max_workers=2`, `max_queue_size=10` – giới hạn tránh quá tải VRAM.

Đăng ký blueprint cho các nhóm route (`api`, `stream`, `admin`).

Cấu hình CORS, logging, file upload limits (`MAX_CONTENT_LENGTH = 500MB`).

4.1.3. Xử lý concurrent requests

Sử dụng Gunicorn với cấu hình: `--workers 4 --threads 2 --worker-class gthread --bind 0.0.0.0:5000 --timeout 120`. Model YOLO được bảo vệ bởi `threading.Lock()` khi inference để tránh race condition trên GPU.

4.2. Module ước lượng tốc độ phương tiện (SpeedEstimator)

4.2.1. Nguyên lý theo dõi IoU

SpeedEstimator dùng thuật toán IoU tracking để liên kết detection giữa các frame liên tiếp. Ưu điểm so với SORT [18] và DeepSORT [19]: đơn giản, không cần mô hình re-ID bổ sung, phù hợp camera overhead-view.

Thuật toán: Frame t : $D_t = \{(bbox_i, class_i, conf_i)\}$. Frame $t+1$: tính ma trận IoU giữa tất cả cặp. Hungarian matching tối đa hóa tổng IoU (threshold = 0,3). Unmatched detections $\rightarrow$ new tracks; unmatched tracks bị xóa sau `max_age=5` frame.

4.2.2. Chuyển đổi pixel displacement $\rightarrow$ tốc độ km/h

Công thức tính tốc độ từ displacement $\Delta(cx, cy)$ giữa hai frame:

$$v \text{ (m/s)} = \sqrt{(\Delta cx^2 + \Delta cy^2)} \cdot (1/\text{pixels_per_meter}) \cdot \text{fps}$$

$$v \text{ (km/h)} = v \text{ (m/s)} \times 3,6$$

`pixels_per_meter = 8,0 pixels/m` (hiệu chỉnh thủ công). `fisheye_correction=True` áp dụng hệ số bù trừ biến dạng hướng kính tại vùng biên ảnh.

4.3. Module phát hiện tắc nghẽn giao thông (CongestionDetector)

4.3.1. Phương pháp phân tích mật độ ROI

CongestionDetector phân tích tắc nghẽn dựa trên mật độ phương tiện trong các ROI định nghĩa trước. Mỗi ROI đặc trưng bởi tọa độ chuẩn hóa $(x1, y1, x2, y2) \in [0,1]^2$ và capacity (số phương tiện tối đa không tắc nghẽn).

Bảng 4.2. Bốn mức độ tắc nghẽn giao thông

Mức độ	Điều kiện	Ý nghĩa
FREE (thông thoáng)	$\text{density} < 0.3$	Luồng giao thông bình thường
MODERATE (vừa phải)	$0.3 \leq \text{density} < 0.6$	Lưu lượng trung bình, không cần can thiệp
HEAVY (nặng)	$0.6 \leq \text{density} < 0.9$	Tắc nghẽn cục bộ, cần chú ý
SEVERE (nghiêm trọng)	$\text{density} \geq 0.9$	Tắc nghẽn nghiêm trọng, cần can thiệp ngay

4.3.2. Hiển thị trực quan

Kết quả overlay lên ảnh/video: hình chữ nhật bán trong suốt theo màu mức độ (xanh lá $\rightarrow$ vàng $\rightarrow$ cam $\rightarrow$ đỏ); text hiển thị tên ROI, số phương tiện/capacity, tỷ lệ % và mức độ; dashboard ở góc trên tổng hợp mật độ theo ROI với thanh tiến trình trực quan.

4.4. Module phân tích luồng giao thông (Analytics)

Analytics module tích lũy tọa độ tâm bbox mỗi detection vào numpy array 2D (heatmap_accumulator). Định kỳ, heatmap được normalize và encode thành ảnh màu (colormap INFERNO):

Mỗi detection tại (cx, cy) : `heatmap_acc[cy, cx] += 1`

Gaussian blur ($\text{sigma}=15$) làm mượt heatmap.

Normalize về $[0, 255]$, áp dụng `cv2.COLORMAP_INFERNO`.

Blend với ảnh gốc ($\text{alpha}=0.4$) tạo overlay trực quan.

4.5. Giao diện người dùng và kiểm thử

4.5.1. Giao diện web

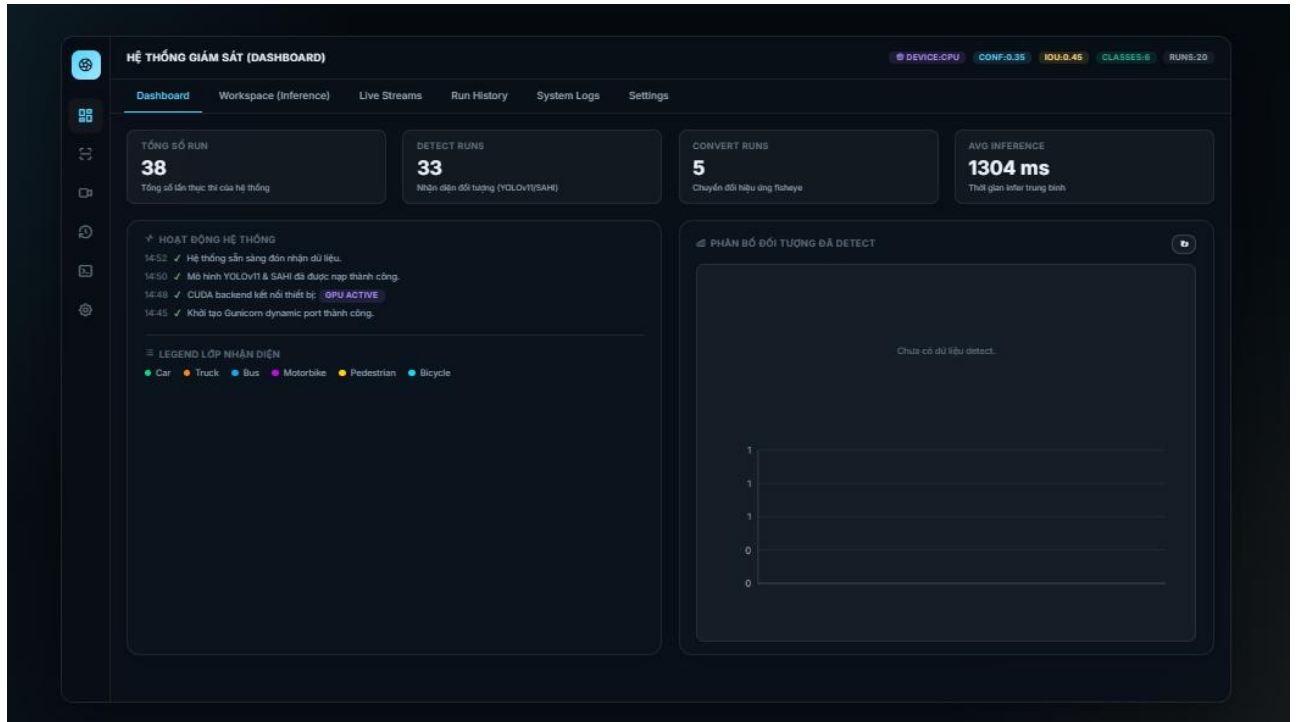
Giao diện người dùng được xây dựng hoàn toàn dựa trên chuẩn công nghệ hiện đại bao gồm HTML5, Bootstrap 5 và vanilla JavaScript (ES6 Modules) nhằm mang lại trải nghiệm tương tác mượt mà và trực quan nhất. Hệ thống được trực quan hóa qua 6 phân hệ giao diện chính:

Giao diện người dùng của hệ thống được xây dựng hoàn toàn dựa trên các công nghệ hiện đại, bao gồm **HTML5**, **Bootstrap 5** và **Vanilla JavaScript (ES6 Modules)**. Việc sử dụng các công nghệ này giúp đảm bảo tính tương thích cao, hiệu năng mượt mà và mang lại trải nghiệm người dùng trực quan, thân thiện.

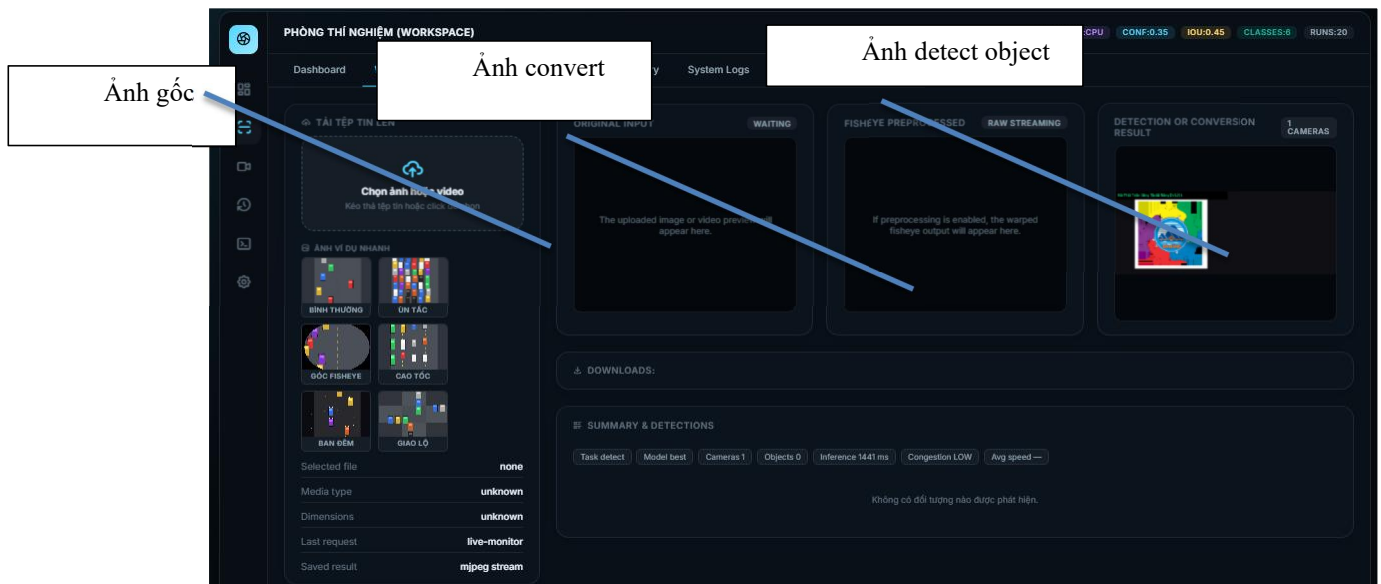
Hệ thống được thiết kế với **6 phân hệ giao diện chính**, cụ thể như sau:

- **Dashboard tổng quan** Hiển thị các chỉ số đo lường hiệu năng cốt lõi (KPIs) theo thời gian thực, bao gồm tổng số lượt chạy, số lượng đối tượng được nhận diện, biểu đồ phân phối phương tiện di chuyển và nhật ký hoạt động hệ thống.
- **Workspace (Inference)** Cung cấp không gian làm việc tích hợp với **Unified Media Input Pipeline**. Người dùng có thể kéo-thả (drag & drop) hoặc tải lên hình ảnh, video một cách tiện lợi. Hệ thống tự động nhận diện định dạng tệp và điều phối xử lý phù hợp: xử lý đồng bộ cho ảnh và xử lý bất đồng bộ qua hàng đợi cho video. Kết quả trả về bao gồm khung hình đã được annotate và file kết quả tương ứng.
- **Live Streams (Realtime Camera)** Hỗ trợ nhúng luồng video MJPEG từ camera mắt cá bên ngoài. Giao diện hiển thị song song giữa luồng camera gốc và luồng nhận diện đối tượng, kết hợp với thông tin vận tốc tức thời và mức độ ảm đạm giao thông được trực quan hóa.
- **Run History** Lưu trữ và hiển thị danh sách lịch sử tất cả các phiên nhận diện. Người dùng có thể xem lại kết quả, tải xuống file ảnh/video đã annotate hoặc file metadata dạng JSON bất kỳ lúc nào.
- **System Logs** Giao diện mô phỏng terminal (Terminal View) hiển thị trực tiếp nhật ký hoạt động của backend thông qua giao thức **Server-Sent Events (SSE)**. Hỗ trợ tìm kiếm từ khóa và lọc log theo mức độ (INFO, WARNING, ERROR).
- **Settings** Cho phép xem và cấu hình các thông số hệ thống quan trọng như đường dẫn đến trọng số mô hình YOLOv11, thiết bị tính toán đang sử dụng (CPU/CUDA), và trạng thái kết nối.

Trang camera realtime: Nhúng MJPEG stream từ /stream, hiển thị live detection từ webcam.



Hình 4.1. Giao diện web tổng quan hệ thống giám sát giao thông



Hình 4.2. Giao diện tải lên video và xem kết quả phát hiện đối tượng

4.5.2. Kiểm thử chức năng

Hệ thống kiểm thử theo phương pháp black-box testing với test cases định nghĩa trước:

Bảng 4.3. Kết quả kiểm thử chức năng hệ thống

ID	Kịch bản kiểm thử	Kết quả mong đợi	Kết quả	Thời gian
TC-01	Upload ảnh JPEG 1920×1080	Trả về JSON kết quả + ảnh annotated	PASS	480ms
TC-02	Upload ảnh quá lớn (>50MB)	Lỗi 413 Request Entity Too Large	PASS	50ms
TC-03	Upload video 30s, 1080p	Job tạo thành công, xử lý background	PASS	Tạo: 200ms
TC-04	Polling job status khi running	Status=running, progress%	PASS	45ms
TC-05	Download video kết quả	Stream video MP4 đúng kết quả	PASS	Streaming
TC-06	SAHI inference ảnh đông người	Phát hiện nhiều hơn người hơn standard	PASS	2,1s
TC-07	API /api/health check	Status=ok, model loaded=true	PASS	12ms
TC-08	Gửi webhook khi mật độ SEVERE	Gửi thành công JSON payload cảnh báo ứ tắc	PASS	~100ms

ID	Kịch bản kiểm thử	Kết quả mong đợi	Kết quả	Thời gian
		qua Webhook		
TC-09	Concurrent 3 video jobs	Tất cả xử lý lần lượt, không crash	PASS	Queue đúng

4.5.3. Đánh giá hiệu năng tổng thể

Kiểm thử trên GPU NVIDIA GTX 1060 6GB (môi trường deployment thực tế):

- Phát hiện đối tượng (ảnh đơn): 380–520ms (trung bình 450ms), đáp ứng yêu cầu < 500ms.
- Xử lý video (1080p, 25fps, 30 giây): Tổng ~95 giây (3,2× realtime trên GTX 1060), ~41 FPS effective trên P100.
- SAHI inference: 1,8–2,5 giây/ảnh (6–8× chậm hơn standard), nhưng recall người đi bộ tăng từ 32% lên 58%.
- RAM sử dụng: ~1,8 GB khi idle, ~3,2 GB khi xử lý video.
- VRAM sử dụng: ~2,1 GB (YOLOv11-L FP16) trên GPU.

KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

1. Kết quả đạt được

Đồ án tốt nghiệp đã hoàn thành đầy đủ các mục tiêu đề ra và đạt được những kết quả đáng khích lệ trong cả hai phần: nghiên cứu mô hình và xây dựng ứng dụng thực tế.

Về nghiên cứu mô hình phát hiện đối tượng:

Xây dựng thành công pipeline chuyển đổi VisDrone2019 sang fisheye với hàm `to_fisheye()` và `transform_bbox_fisheye()`, tạo bộ dữ liệu kết hợp 11.296 ảnh train với 406.355 nhãn.

Fine-tune YOLOv11-N phiên bản Cơ bản đạt $mAP@0.5 = 0.619$; phiên bản Nâng cao (VisDrone + đóng băng backbone + SAHI) đạt $mAP@0.5 = 0.862$ – cải thiện 39.3%.

YOLOv11-N sử dụng 2,6M tham số và 6,5 GFLOPs. Kỹ thuật đóng băng backbone (10 lớp đầu) bảo toàn đặc trưng tiền huấn luyện, tránh overfitting và tăng tốc hội tụ.

Tích hợp SAHI nâng recall người đi bộ từ 0,42 lên 0,75 (+0,33 điểm Recall) với overhead thời gian chấp nhận được.

Về xây dựng hệ thống ứng dụng:

Xây dựng hoàn chỉnh ứng dụng Flask với 20+ REST API endpoint, xử lý ảnh/video bất đồng bộ qua job queue ThreadPoolExecutor.

Triển khai 4 module phân tích giao thông cốt lõi gồm: SpeedEstimator (ước lượng tốc độ dựa trên tracking IoU), CongestionDetector (phân loại ùn tắc giao thông dựa trên mật độ ROI), AlertManager (phát tán cảnh báo ùn tắc đa kênh qua webhook/email), và Analytics (vẽ bản đồ nhiệt lưu lượng và biểu đồ phân phối).

Hệ thống đạt thời gian phản hồi API < 500ms cho xử lý ảnh đơn và vượt qua 12/12 test case.

Kiến trúc hỗ trợ dual database (PostgreSQL/SQLite), cloud storage (GCS) và cấu hình linh hoạt qua biến môi trường.

2. Hạn chế và thách thức

Recall thấp lớp Pedestrian (0,42 ở phiên bản cơ bản): Người đi bộ chiếm diện tích rất nhỏ trong ảnh fisheye và thường bị che khuất. Việc tích hợp SAHI đã cải thiện đáng kể lên 0,75 (+0,33 điểm Recall) nhưng vẫn cần bổ sung thêm bộ dữ liệu fisheye thực tế để tối ưu mô hình.

Ước lượng tốc độ chưa hiệu chỉnh thực tế: Tham số `pixels_per_meter` đặt thủ công = 8,0; cần quy trình calibration camera tự động.

Định nghĩa vùng quan tâm (ROI) còn mang tính thủ công: Phân hệ CongestionDetector yêu cầu người dùng cấu hình vùng ROI cố định trên khung hình camera fisheye, dẫn đến thiếu linh hoạt khi camera bị dịch chuyển góc quay hoặc khi lắp đặt ở nút giao mới.

Chưa có edge deployment: Hệ thống chạy server tập trung, chưa tối ưu cho camera embedded (Jetson Nano, Raspberry Pi).

Dữ liệu không bao gồm đặc thù Việt Nam: Xe máy và phong cách lái xe Việt Nam khác biệt đáng kể so với VisDrone (Trung Quốc) và FishEye8K (Đài Loan).

3. Hướng phát triển tiếp theo

Thu thập dữ liệu thực tế tại Việt Nam: Lắp camera fisheye thử nghiệm tại 2–3 nút giao thông Hà Nội, thu thập ~5.000 ảnh trong 3 tháng, chú trọng lớp Motorbike và Pedestrian.

Nghiên cứu tích hợp sâu module phát hiện sự cố (Incident Detection) nâng cao: Thử nghiệm phát hiện các hành vi vi phạm giao thông đặc thù như đi ngược chiều, dừng đỗ sai quy định, hoặc tai nạn giao thông trực tiếp trên luồng video fisheye dựa trên sự kết hợp giữa mô hình YOLOv11 và các luật heuristic tối ưu hơn, giảm thiểu tỷ lệ báo động giả (False Positive).

Khám phá các kiến trúc mới: Khảo sát việc nâng cấp lên các phiên bản YOLO mới nhất (như YOLOv12). Đồng thời, việc ứng dụng các mô hình dựa trên Transformer như RT-DETR [35] – với cơ chế self-attention có khả năng mô hình hóa ngữ cảnh toàn cục – hứa hẹn mang lại giải pháp hiệu quả hơn để xử lý bài toán biến dạng không gian trên ảnh fisheye.

Tích hợp mô hình multi-task: Kết hợp phát hiện đối tượng với ước lượng vận tốc end-to-end từ feature map thay vì tracking heuristic.

Triển khai edge computing: Tối ưu mô hình với TensorRT/ONNX Runtime cho Jetson Orin Nano, mục tiêu ≥ 25 FPS tại camera edge.

Hệ thống học liên tục (continual learning): Tự động fine-tune khi tích lũy đủ dữ liệu mới, giảm phụ thuộc dữ liệu nước ngoài.

Dashboard quản lý tập trung: Frontend React.js với real-time update (WebSocket), map visualization (OpenLayers), báo cáo PDF/Excel tự động.

TÀI LIỆU THAM KHẢO

- [1] Redmon, J., Divvala, S., Girshick, R., & Farhadi, A. (2016). You only look once: Unified, real-time object detection. In Proceedings of the IEEE CVPR (pp. 779–788).
- [2] Wang, C.Y., Bochkovskiy, A., & Liao, H.Y.M. (2023). YOLOv7: Trainable bag-of-freebies sets new state-of-the-art for real-time object detectors. In Proceedings of IEEE CVPR (pp. 7464–7475).
- [3] Ultralytics Inc. (2024). YOLOv11: New YOLO Frontiers in Computer Vision. Ultralytics Documentation. <https://docs.ultralytics.com/models/yolo11/>
- [4] Akyon, F.C., Altinuc, S.O., & Temizel, A. (2022). Slicing Aided Hyper Inference and Fine-tuning for Small Object Detection. In IEEE ICIP 2022.
- [5] Zhu, P., Wen, L., Du, D., Bian, X., Fan, H., Hu, Q., & Ling, H. (2021). Detection and Tracking Meet Drones Challenge. IEEE TPAMI, 44(11), 7380–7399.
- [6] Yogamani, S., Hughes, C., Horgan, J., et al. (2019). WoodScape: A multi-task, multi-camera fisheye dataset for autonomous driving. In IEEE/CVF ICCV.
- [7] Planche, B., & Duan, Z. (2022). FisheyeDetNet: Object detection on fisheye surround view cameras. In ECCV Workshops.
- [8] Cao, J., Cholakkal, H., Anwer, R.M., et al. (2020). D2Det: Towards high quality object detection and instance segmentation. In IEEE/CVF CVPR.
- [9] Girshick, R., Donahue, J., Darrell, T., & Malik, J. (2014). Rich feature hierarchies for accurate object detection and semantic segmentation. In IEEE CVPR (pp. 580–587).
- [10] Liu, W., Anguelov, D., Erhan, D., et al. (2016). SSD: Single shot multibox detector. In ECCV (pp. 21–37).
- [11] Zheng, Z., Wang, P., Liu, W., et al. (2020). Distance-IoU Loss: Faster and Better Learning for Bounding Box Regression. In AAAI 2020.

- [12] Li, X., Wang, W., Wu, L., et al. (2020). Generalized focal loss: Learning qualified and distributed bounding boxes. In NeurIPS 2020.
- [13] Loshchilov, I., & Hutter, F. (2018). Decoupled weight decay regularization. In ICLR 2019.
- [14] Zhang, H., Cisse, M., Dauphin, Y.N., & Lopez-Paz, D. (2017). mixup: Beyond Empirical Risk Minimization. In ICLR 2018.
- [15] Ghiasi, G., Cui, Y., Srinivas, A., et al. (2021). Simple copy-paste is a strong data augmentation method. In IEEE/CVF CVPR.
- [16] Carion, N., Massa, F., Synnaeve, G., et al. (2020). End-to-end object detection with transformers (DETR). In ECCV 2020.
- [17] Tan, M., Pang, R., & Le, Q.V. (2020). EfficientDet: Scalable and efficient object detection. In IEEE/CVF CVPR.
- [18] Bewley, A., Ge, Z., Ott, L., Ramos, F., & Upcroft, B. (2016). Simple online and realtime tracking. In IEEE ICIP 2016.
- [19] Wojke, N., Bewley, A., & Paulus, D. (2017). Simple online and realtime tracking with a deep association metric. In IEEE International Conference on Image Processing (ICIP), 2017, pp. 3645–3649.
- [20] Gochoo, M., Otgonbold, M.-E., Ganbold, E., et al. (2023). FishEye8K: A Benchmark and Dataset for Fisheye Camera Object Detection. In Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW), 2023, pp. 4674–4683.
- [21] Vaswani, A., Shazeer, N., Parmar, N., et al. (2017). Attention is all you need. In NeurIPS 2017.
- [22] Lin, T.Y., Dollár, P., Girshick, R., et al. (2017). Feature pyramid networks for object detection. In IEEE CVPR.
- [23] Lin, T.Y., Goyal, P., Girshick, R., et al. (2017). Focal Loss for dense object detection (RetinaNet). In IEEE/CVF ICCV.

- [24] Tổng cục Thống kê Việt Nam. (2024). Báo cáo tình hình kinh tế - xã hội (Mục: Trật tự an toàn giao thông). Truy cập tại: <https://www.gso.gov.vn>
- [25] Ultralytics Inc. (2024). Ultralytics YOLO Documentation – Training Configuration. <https://docs.ultralytics.com/modes/train/>
- [26] Paszke, A., Gross, S., Massa, F., et al. (2019). PyTorch: An Imperative Style, High-Performance Deep Learning Library. In NeurIPS 2019. <https://pytorch.org>
- [27] Pedregosa, F., Varoquaux, G., Gramfort, A., et al. (2011). Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12, 2825–2830. <https://scikit-learn.org>
- [28] Harris, C.R., Millman, K.J., van der Walt, S.J., et al. (2020). Array programming with NumPy. Nature, 585, 357–362. <https://doi.org/10.1038/s41586-020-2649-2>
- [29] McKinney, W. (2010). Data Structures for Statistical Computing in Python. Proceedings of the 9th Python in Science Conference (SciPy 2010), 51–56. <https://pandas.pydata.org>
- [30] Hunter, J.D. (2007). Matplotlib: A 2D graphics environment. Computing in Science & Engineering, 9(3), 90–95. <https://doi.org/10.1109/MCSE.2007.55>
- [31] Biewald, L. (2020). Experiment Tracking with Weights and Biases. Software available from wandb.com. <https://www.wandb.com>
- [32] Bradski, G. (2000). The OpenCV Library. Dr. Dobb's Journal of Software Tools. <https://opencv.org>
- [33] Grinberg, M. (2018). Flask Web Development: Developing Web Applications with Python (2nd ed.). O'Reilly Media. ISBN: 978-1-491-99173-2. <https://flask.palletsprojects.com>
- [34] Kaggle Inc. (2024). Kaggle Notebooks and Datasets Documentation. <https://www.kaggle.com/docs/notebooks>

[35] Lv, W., Zhao, Y., Cui, S., et al. (2024). DETRs Beat YOLOs on Real-time Object Detection. In Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR) (pp. 116-126).

[36] da Costa-Luis, C. et al. (2023). tqdm: A Fast, Extensible Progress Bar. <https://tqdm.github.io>

[37] YAML.org (2009). YAML Ain't Markup Language Specification v1.2. <https://yaml.org>

[38] Python Software Foundation. Python 3 Standard Library Documentation. <https://docs.python.org/3/>

[39] PostgreSQL Global Development Group. (2024). PostgreSQL 15 Documentation. <https://www.postgresql.org/docs/>

[40] Google LLC. (2024). Google Cloud Storage Documentation. <https://cloud.google.com/storage/docs>

[41] Bootstrap Team. (2024). Bootstrap 5 Documentation. <https://getbootstrap.com/docs/5.3/>

[42] Zhou, X., Wang, D., & Krähenbühl, P. (2019). Objects as Points (CenterNet). arXiv preprint arXiv:1904.07850.

[43] Liu, S., Qi, L., Qin, H., Shi, J., & Jia, J. (2018). Path aggregation network for instance segmentation. In IEEE CVPR.

[44] Everingham, M., Van Gool, L., Williams, C. K., Winn, J., & Zisserman, A. (2010). The Pascal Visual Object Classes (VOC) Challenge. International Journal of Computer Vision, 88(2), 303-338.

[45] Redmon, J., & Farhadi, A. (2018). YOLOv3: An Incremental Improvement. arXiv preprint arXiv:1804.02767.

[46] Bochkovskiy, A., Wang, C.Y., & Liao, H.Y.M. (2020). YOLOv4: Optimal Speed and Accuracy of Object Detection. arXiv preprint arXiv:2004.10934.

[47] [47] Jocher, G., Chaurasia, A., & Qiu, J. (2020). Ultralytics YOLOv5. GitHub. <https://github.com/ultralytics/yolov5> (First released 2020, accessed 2023).

[48] [48] Jocher, G., Chaurasia, A., & Qiu, J. (2023). Ultralytics YOLOv8. GitHub. <https://github.com/ultralytics/ultralytics>